

学校代码: 10284
分类号: O643.12
密 级: 公开
U D C: 544.4
学 号: 502023330078



南京大學

硕士学位论文

论文题目	基于大语言模型的 软件漏洞检测 关键技术研究
作者姓名	臧正贻
专业名称	计算机
研究方向	软件工程
导师姓名	王林章 王豫

2026 年 4 月 1 日

答辩委员会主席_____

评 阅 人_____

论文答辩日期 2026 年 5 月 21 日

研究生签名:

导师签名:

Key Technologies of Software Vulnerability Detection Based on Large Language Models

by
TSANG CHENGYI

Supervised by
Wang LinZhang
Wang Yu

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of

MASTER

in

Computer Science



School of Computer Science
Nanjing University

April 1, 2026

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：基于大语言模型的软件漏洞检测关键技术研究

计算机 专业 2026 级硕士生姓名：臧正贻

指导教师（姓名、职称）：王林章 王豫

摘 要

软件漏洞检测是软件安全领域的关键问题。传统方法在误报控制与跨场景泛化方面存在明显不足，而现有深度学习方法在复杂漏洞语义建模、持续学习能力及对数据质量的依赖等方面仍面临瓶颈。围绕上述挑战，针对大语言模型驱动的漏洞检测方法展开系统研究。

针对复杂漏洞语义难以有效建模的问题，构建了结合静态分析的上下文增强检测方法。通过精确提取与漏洞相关的关键上下文信息，引导大语言模型聚焦于语义关键区域，从而在降低计算开销的同时提升检测有效性。在空指针异常（Null Pointer Exception, NPE）任务上的结果表明，该方法能够准确识别漏洞并刻画缺陷传播路径，F1 分数达到 0.934，体现出良好的检测能力与实用性。

针对模型在持续演进过程中易发生灾难性遗忘的问题，设计了基于核心集的增量学习机制。利用参数变化特征刻画数据语义分布，并结合层次聚类构建高代表性的核心子集，在引入新知识的同时保持既有能力。实验表明，仅使用约 20% 的核心数据即可获得接近全量训练的性能，同时训练时间从 22 小时降低至约 12 小时，显著提升了模型更新效率。

针对漏洞数据质量参差不齐对模型性能的制约，构建了基于模型行为分析的数据精炼方法。通过识别并剔除有害或低质量样本，实现训练数据的动态优化。实验结果显示，经数据精炼后，在多个开源数据集上的 F1 分数平均提升约 5 个百分点，且在低质量场景下性能增益更为显著。

关键词：软件漏洞检测；大语言模型；增量学习；数据精炼；程序切片

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Key Technologies of Software Vulnerability DetectionBased on Large Language Models

SPECIALIZATION: Computer Science

POSTGRADUATE: TSANG CHENGYI

MENTOR: Wang LinZhang Wang Yu

ABSTRACT

Software vulnerability detection is a fundamental problem in the field of software security. Traditional approaches suffer from high false positive rates and limited cross-scenario generalization, while existing deep learning-based methods still face significant challenges in modeling complex vulnerability semantics, supporting continual learning, and handling data quality issues. To address these challenges, a systematic study is conducted on large language model (LLM)-driven vulnerability detection.

To address the difficulty of modeling complex vulnerability semantics, a context-enhanced detection approach integrating static analysis is developed. By precisely extracting vulnerability-relevant contextual information, the method guides the LLM to focus on semantically critical regions, thereby improving detection effectiveness while reducing computational overhead. Experimental results on the Null Pointer Exception (NPE) task demonstrate that the approach can accurately identify vulnerabilities and characterize defect propagation paths, achieving an F1 score of 0.934, indicating strong effectiveness and practical applicability.

To mitigate catastrophic forgetting during continuous model evolution, a core-set-based incremental learning mechanism is designed. Parameter variation features are leveraged to characterize the semantic distribution of data, and hierarchical clustering is employed to construct a highly representative core subset. This enables the model to retain previously acquired knowledge while incorporating new information. Experimental results show that using only approximately 20% of the data achieves performance comparable to full-data training, while reducing training time from 22 hours

to approximately 12 hours, significantly improving update efficiency.

To address the impact of heterogeneous data quality on model performance, a data refinement method based on model behavior analysis is proposed. By identifying and filtering harmful or low-quality samples, the approach enables continuous optimization of training data. Experimental results indicate that, after refinement, the F1 score improves by approximately 5 percentage points on average across multiple open-source datasets, with more pronounced gains observed on low-quality datasets.

KEYWORDS: Software Vulnerability Detection; Large Language Model; Incremental Learning; Data Refinement; Program Slicing

目 录

目 录	V
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	3
1.2.1 基于深度学习的漏洞检测研究现状	3
1.2.2 大语言模型在代码任务中的应用研究	4
1.2.3 增量学习与灾难性遗忘问题研究	5
1.2.4 数据质量与数据精炼研究	5
1.3 研究内容与主要贡献	6
1.3.1 设计基于大语言模型的漏洞检测方法	6
1.3.2 提出基于核心集的增量学习方法	7
1.3.3 设计基于数据精炼的质量提升方法	8
1.4 实验验证	8
1.5 论文组织结构	8
1.6 本章小结	9
第二章 相关技术与研究现状	11
2.1 引言	11
2.2 软件漏洞检测技术	11
2.2.1 传统漏洞检测方法	11
2.2.2 基于深度学习的漏洞检测	12
2.2.3 漏洞检测方法分类	13
2.3 大语言模型基础	13
2.3.1 Transformer 架构与代码预训练	13

2.3.2	大语言模型的微调	14
2.4	增量学习与灾难性遗忘	15
2.4.1	灾难性遗忘问题	15
2.4.2	增量学习方法分类	15
2.4.3	大语言模型的增量学习	17
2.5	数据质量与数据精炼	17
2.5.1	数据质量对模型性能的影响	17
2.5.2	数据精炼技术	18
2.6	程序切片技术	18
2.6.1	程序切片的基本概念	18
2.6.2	程序依赖图	19
2.6.3	切片在漏洞检测中的应用	19
2.7	本章小结	20
第三章 基于大语言模型的漏洞检测方法设计		21
3.1	引言	21
3.2	整体方法框架	21
3.3	模型选择与优化	22
3.3.1	基座模型选择	22
3.3.2	QLoRA 微调技术	23
3.4	上下文构建方法	24
3.4.1	Source 锚点识别	24
3.4.2	函数级上下文提取	24
3.4.3	语句级上下文筛选	25
3.4.4	上下文组合策略	25
3.5	多级评估体系	26
3.5.1	评估需求分析	26
3.5.2	三级评估指标	26
3.5.3	评估流程与指标计算	27
3.5.4	评估方法的优势	28

3.6 本章小结	28
第四章 基于核心集的增量学习方法	31
4.1 引言	31
4.2 灾难性遗忘问题分析	32
4.2.1 遗忘现象的观察	32
4.2.2 理论分析	32
4.2.3 增量学习的挑战	32
4.3 方法概述	32
4.4 参数变化特征提取	33
4.4.1 核心洞察	33
4.4.2 特征提取流程	34
4.4.3 与梯度特征的比较	34
4.5 层次聚类核心集选择	35
4.5.1 层次聚类算法	35
4.5.2 核心集采样策略	35
4.5.3 核心集大小的选择	35
4.5.4 核心集的多样性表征	36
4.6 混合相似度算法	36
4.6.1 动机	36
4.6.2 混合相似度设计	36
4.7 模型更新策略	37
4.8 本章小结	37
第五章 基于数据精炼的质量提升方法	39
5.1 引言	39
5.2 方法概述	39
5.3 有害数据识别	40
5.3.1 验证集错误分析	40
5.3.2 困惑度分析	42
5.4 聚类分布过滤	42

5.4.1	聚类分析	42
5.4.2	有害密度计算	43
5.4.3	分层过滤策略	43
5.5	精炼数据集构建	43
5.5.1	精炼数据集组成	43
5.6	核心集与精炼的结合	44
5.6.1	动机	44
5.6.2	结合流程	44
5.6.3	实验结果	44
5.7	本章小结	45
第六章 实验设计与结果分析		47
6.1	引言	47
6.2	实验环境	47
6.2.1	硬件环境	47
6.2.2	软件环境	47
6.3	数据集	48
6.3.1	NPE-withpath 数据集	48
6.3.2	Big-Vul 数据集	48
6.3.3	PrimeVul 数据集	48
6.3.4	Devign 数据集	48
6.4	评估指标	49
6.4.1	基础分类指标	49
6.4.2	多级评估指标	49
6.5	实验一：基础检测实验	49
6.5.1	实验设置	49
6.5.2	结果与分析	50
6.6	实验二：增量学习实验	52
6.6.1	实验设置	52
6.6.2	结果与分析	53

6.7	实验三：数据精炼实验	56
6.7.1	实验设置	57
6.7.2	结果与分析	57
6.8	威胁效度分析	59
6.8.1	内部效度	59
6.8.2	外部效度	60
6.9	本章小结	60
第七章 总结与展望		61
7.1	研究工作总结	61
7.2	研究创新点	61
7.3	研究局限性	62
7.4	未来研究方向	62
7.5	结语	63
参考文献		65
致 谢		71
附录 A 附录		73
A.1	实验数据补充	73
A.2	源代码说明	73

第一章 绪论

1.1 研究背景与意义

软件漏洞检测在软件安全领域中占据着至关重要的地位。随着信息技术的飞速发展，软件系统已经渗透到社会运行的各个角落，从金融交易、医疗健康到国防安全、基础设施控制，软件的质量与安全性直接关系到国家安全、经济发展和社会稳定。软件漏洞作为软件系统中普遍存在的安全隐患，可能被恶意攻击者利用来窃取敏感信息、破坏系统功能或实施其他恶意行为，造成难以估量的经济损失和社会影响。

根据美国国家标准与技术研究院（NIST）的统计数据显示，软件漏洞的数量呈逐年上升趋势，仅 2023 年披露的公共漏洞数量就超过了 28000 个，创历史新高。这些漏洞涵盖了缓冲区溢出、空指针解引用、资源泄漏、SQL 注入等多种类型，其中 C/C++ 语言编写的系统软件由于直接操作底层内存，更容易出现内存安全相关的漏洞。因此，如何高效、准确地检测软件漏洞，在漏洞被利用之前进行修复和缓解，已成为软件安全领域亟待解决的核心问题。

传统的软件漏洞检测方法主要包括静态分析工具、动态分析工具以及人工代码审查等。静态分析工具如 Cppcheck^[1]、Clang Static Analyzer^[2]、Infer^[3] 等，通过预定义的规则模式对源代码进行扫描，能够在不执行代码的情况下发现潜在的安全问题。这类方法的优点是可以覆盖较大的代码规模，但存在明显的局限性：一方面，基于规则的方法需要专家手动编写和维护检测规则，随着新漏洞类型的不断出现，规则库需要持续更新，消耗大量人工成本；另一方面，静态分析工具往往产生较高的误报率和漏报率，尤其是在处理复杂的跨过程漏洞时效果不佳。

动态分析方法通过实际执行或模拟执行程序来检测漏洞，包括模糊测试（Fuzzing）、符号执行等技术。模糊测试通过向程序输入大量随机或半随机的测试数据，观察程序是否出现崩溃、内存泄漏等异常行为来发现漏洞。符号执行

则将程序输入符号化，通过路径约束求解来探索程序的执行路径。这些方法虽然在某些场景下表现良好，但存在测试覆盖率有限、计算开销大、难以处理大规模程序等问题。

人工代码审查是最直接的漏洞检测方式，由经验丰富的安全专家对代码进行逐行检查。这种方法理论上可以发现各种类型的漏洞，但高度依赖审查者的专业知识与经验，且在面对数百万行代码的大型软件项目时效率极低，难以满足现代软件开发快速迭代的需求。

近年来，随着深度学习技术的快速发展，基于深度学习的软件漏洞检测方法逐渐受到学术界和工业界的广泛关注。与传统方法相比，深度学习模型能够通过训练自动学习代码中的漏洞模式，无需人工设计复杂的检测规则，具有更强的适应性和泛化能力。尤其是大语言模型（Large Language Model, LLM）的出现，为软件漏洞检测带来了新的机遇。大语言模型在海量代码语料上进行了预训练，已经学习到了丰富的代码语义知识和编程模式，展现出对代码复杂语义的强理解能力。通过指令微调（Instruction Tuning）、QLoRA 等高效参数微调技术，将预训练的大语言模型适配到漏洞检测任务上，可以有效避免从头训练模型的高昂成本，同时获得更好的检测性能。

近期，基于大语言模型的 AI Agent 技术在代码分析与安全检测领域展现出显著潜力。以 Claude Code 为代表的工具能够自主调用静态分析、符号执行、动态测试等专业化工具，形成多工具协同的漏洞检测与修复 workflow，其表现已接近经验丰富的安全工程师。然而，此类基于 Agent 的方法在实际应用中仍面临若干挑战：首先，多轮迭代与工具调用导致 Token 消耗量巨大，显著推高使用成本；其次，分析过程耗时较长，难以满足软件开发快速反馈的需求；再次，主流方案依赖闭源商业模型，存在数据隐私与供应链安全风险。上述因素限制了 Agent 技术在大规模安全检测场景中的普及应用。

针对上述问题，探索了一条更为高效、经济的技术路径：采用参数规模适中的开源大语言模型，结合轻量级静态分析工具，实现高精度、低成本的漏洞检测。核心思路在于，充分利用静态分析工具在代码切片与上下文限定方面的精确性，通过过程间切片技术构建能够覆盖完整缺陷传播路径的代码上下文，使模型输入尽可能涵盖与缺陷分析相关的全部语义信息；同时，设计结构化的输出格式，要求模型明确给出从缺陷触发点（Source）到缺陷影响点（Sink）的完

整数据流路径，而非仅输出二元判定结果。这种设计既使模型能够在完整的缺陷上下文中进行推理，提升了检测的准确性和可解释性，又使输出结果能够直接被开发者理解和用于漏洞修复，降低了人工验证成本。

此外，软件漏洞检测技术的实际落地还面临两个关键问题。其一，软件系统处于持续演进之中，编程语言特性、硬件架构和应用场景的变化可能引入新的漏洞类型，检测工具需要具备持续学习新知识的能力。然而，深度学习模型普遍存在“灾难性遗忘”问题，即在学习新知识时会遗忘旧知识，导致原有检测能力下降。如何在不损害既有性能的前提下实现检测模型的增量更新，是需要解决的重要问题。其二，现有公开漏洞检测数据集普遍存在标注质量低、漏洞类型单一、正负样本不平衡等问题，直接使用此类数据训练的模型性能难以满足实际需求。因此，如何从低质量数据中精炼出高质量训练样本，提升模型训练效果，同样是需要深入研究的问题。

基于上述分析，围绕基于大语言模型的软件漏洞检测技术展开系统探索，旨在构建一个高效、准确、可持续演进的漏洞检测框架。相关研究得到了企业合作项目的支持，为后续应用转化奠定了基础。

1.2 国内外研究现状

1.2.1 基于深度学习的漏洞检测研究现状

基于深度学习的漏洞检测研究可以追溯到 2018 年，Li 等人提出的 VulDeeP-ecker^[4]是最早将深度学习应用于漏洞检测的代表性工作之一。该方法将代码表示为数据依赖图（Data Dependency Graph），利用双向 LSTM 网络学习代码特征，能够检测缓冲区溢出和缓冲区读取漏洞。随后，Lin 等人提出了跨项目的迁移表示学习方法，通过在不同项目之间迁移漏洞知识来提高检测的泛化能力^[5]。

随着 Transformer 架构的兴起，基于预训练语言模型的漏洞检测方法逐渐成为主流。Choi 等人利用神经记忆网络从原始代码中预测缓冲区溢出漏洞，实现了端到端的漏洞检测^[6]。Devign 是另一个具有影响力的工作，该研究从四个大型 C 语言开源项目（Linux Kernel、QEMU、Wireshark、FFmpeg）中收集并人工标注了函数级漏洞数据，提出了基于图神经网络的漏洞检测方法^[7]。

近年来，研究者开始探索利用大语言模型进行漏洞检测。CodeBERT、Graph-

CodeBERT 等代码预训练模型在代码理解任务上展现出强大能力，为漏洞检测提供了新的技术路线。这些模型在海量代码语料上进行了预训练，学习到了代码的语法结构、语义信息以及数据流和控制流特征，能够有效捕捉漏洞的深层模式。

1.2.2 大语言模型在代码任务中的应用研究

大语言模型在代码任务中的应用研究是当前的热点方向。StarCoder、CodeLlama、CodeQwen 等代码专用大语言模型相继发布，在代码生成、代码补全、代码修复等任务上取得了突破性进展。Lozhkov 等人发布的 StarCoder2 模型在多个代码基准测试中达到了当时的最优水平，展现了大语言模型处理复杂代码任务的能力^[8]。

在漏洞检测方面，大语言模型的优势主要体现在以下几个方面：第一，大语言模型能够直接处理原始代码序列，无需像小模型那样将代码转换为控制流图、程序依赖图等中间表示形式，减少了预处理开销和误差引入；第二，大语言模型参数量大，能够捕捉更复杂的代码模式和依赖关系，具有较强的泛化能力；第三，大语言模型通常支持较长的输入序列，可以覆盖更完整的代码上下文，有利于检测跨函数、跨模块的复杂漏洞；第四，大语言模型已经通过预训练学习到了海量代码知识，微调时只需学习漏洞检测的特定知识，训练效率高。

近期，基于大语言模型的 AI Agent 在代码任务上的进展尤为引人注目。这类 Agent 系统不仅能够独立理解和分析代码，还能主动调用静态分析工具（如 Cppcheck、Infer）、符号执行引擎（如 KLEE）、动态测试框架（如 AFL、libFuzzer）等专业工具进行协同分析。通过将复杂任务分解为多个子任务、智能编排工具调用、并根据中间结果进行迭代推理，Agent 系统能够处理传统单一方法难以应对的复杂漏洞检测场景。例如，在检测到潜在漏洞后，Agent 可以自动调用符号执行工具验证路径可行性，或调用动态测试工具生成复现用例，最后综合各类证据给出漏洞判定和修复建议。这种多工具协同的工作模式显著提升了漏洞检测的准确性和可信度，已接近人类安全专家的分析能力。

1.2.3 增量学习与灾难性遗忘问题研究

灾难性遗忘 (Catastrophic Forgetting) 是深度学习模型在持续学习过程中面临的普遍问题。Toneva 等人通过实证研究发现, 深度神经网络在学习新数据时会遗忘之前学习过的样本, 导致模型在旧任务上的性能显著下降^[9]。Wang 等人对持续学习进行了全面综述, 将现有方法分为基于回放、基于正则化和基于参数隔离三大类^[10]。

基于回放 (Replay) 的方法通过保留一部分旧数据或使用生成模型合成旧数据的代表样本, 在训练新任务时同时训练旧数据, 从而缓解遗忘问题。Aljundi 等人提出了基于梯度的样本选择方法, 通过计算样本梯度之间的余弦相似度来选择代表性样本^[11]。Tiwari 等人提出了梯度核心集 (Gradient Coreset) 方法, 将核心集选择问题转化为优化问题, 在视觉效果和效率之间取得平衡^[12]。

然而, 现有的增量学习方法在处理大语言模型时面临新的挑战。大语言模型参数量巨大, 存储每个样本的梯度变化计算开销和存储开销都非常高, 难以直接应用于海量参数的模型。此外, 如何确定核心集的大小、如何平衡新旧数据的比例等问题也缺乏系统的研究。

1.2.4 数据质量与数据精炼研究

数据质量对深度学习模型的性能有着决定性影响。Jain 等人对 C/C++ 漏洞检测数据集进行了系统性评估, 发现现有数据集普遍存在标注质量低、漏洞类型单一、正负样本不平衡等问题^[13]。这些问题直接影响了模型的训练效果和泛化能力。

数据精炼 (Data Refinement) 是提升数据质量的重要手段, 主要包括数据清洗、数据增强、困难样本挖掘等方法。有害数据过滤是数据精炼的关键环节, 通过识别并剔除对模型训练产生负面影响的数据, 可以有效提升模型的收敛速度和最终性能。困惑度 (Perplexity) 是衡量模型对数据拟合程度的重要指标, 高困惑度的数据通常表示模型难以理解或存在标注错误, 可作为筛选困难样本的依据。

在漏洞检测领域, 数据精炼研究相对较少, 现有工作主要关注数据增强和样本平衡, 对于如何从低质量数据集中精炼出高质量训练数据的研究仍有待深入。

1.3 研究内容与主要贡献

四项主要工作之间存在紧密的逻辑关联：首先设计基于大语言模型的漏洞检测方法实现高效检测，接着提出增量学习方法解决模型持续演进问题，再设计数据精炼方法进一步提升数据质量。三项核心研究内容的关系如图1-1所示。

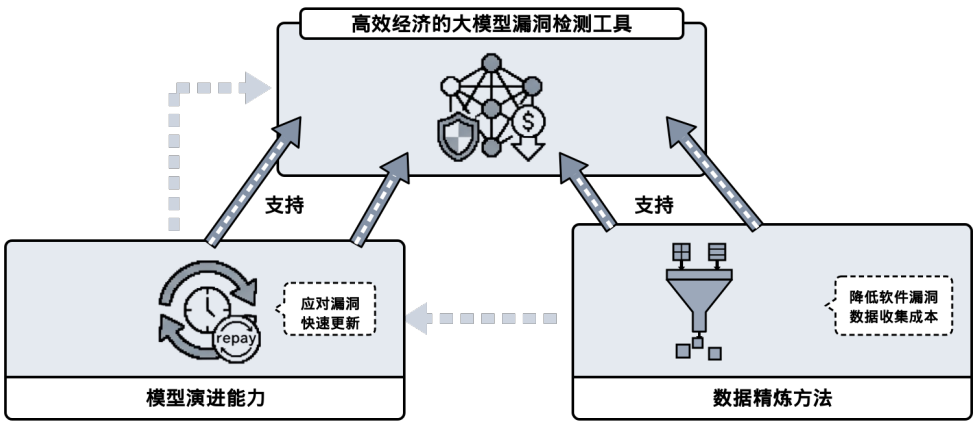


图 1-1 三项核心研究内容关系图

针对现有研究存在的挑战，围绕基于大语言模型的软件漏洞检测技术展开系统研究，主要研究内容和贡献如下：

1.3.1 设计基于大语言模型的漏洞检测方法

针对传统小模型对复杂漏洞语义建模能力不足的问题，采用大语言模型 CodeQwen2.5-7B 作为基座模型，设计了一套完整的漏洞检测方案。核心思路在于，充分利用静态分析工具在代码切片与上下文限定方面的精确性，通过过程间切片技术构建能够覆盖完整缺陷传播路径的代码上下文，使模型输入尽可能涵盖与缺陷分析相关的全部语义信息；同时，设计结构化的输出格式，要求模型明确给出从缺陷触发点（Source）到缺陷影响点（Sink）的完整数据流路径，而非仅输出二元判定结果。这种设计既使模型能够在完整的缺陷上下文中进行推理，提升了检测的准确性和可解释性，又使输出结果能够直接被开发者理解和用于漏洞修复，降低了人工验证成本。

(1)模型选择与优化:综合考虑模型性能与硬件资源限制,选择 CodeQwen2.5-7B-chat 作为基座模型。采用 QLoRA (Quantized Low-Rank Adaptation) 微调技术,通过量化和秩分解大幅降低显存占用,在单卡 H800 上支持 8192 的最大上下文

长度。

(2) 轻量级切片方法：设计了两级切片策略——基于调用图的粗粒度切片和基于程序依赖图（PDG）的细粒度切片。调用图切片通过广度优先搜索确定与 Source 语句相关的函数集合，PDG 切片进一步过滤无关代码，仅保留存在数据依赖和控制依赖的语句。两种切片方法可结合使用，在计算开销和切片质量之间取得平衡。通过静态分析工具的精确切片，模型只需关注与漏洞相关的代码片段，大幅减少了 Token 消耗和推理时间。

(3) 任务形式设计：将传统漏洞检测任务重新形式化为“给定 Source，判断是否存在可达 Sink”的数据流判断任务。消融实验表明，不同的任务形式变体（是否要求输出路径）对检测性能影响较小（F1 分数在 0.934-0.948 之间）。性能提升的核心在于通过过程间切片将能够覆盖完整缺陷上下文的代码段作为输入，而非任务形式的具体设计。但路径信息对于用户使用模型给出的结果具有重要价值——它可以帮助开发者快速定位漏洞传播路径，理解漏洞成因，从而更高效地进行修复。

(4) 多级评估体系：提出三级评估指标，一级评估检测模型是否正确预测缺陷存在及 Source 位置，二级评估检查预测的 Sink 语句是否与 Source 存在依赖关系，三级评估验证从 Source 到 Sink 的依赖传递顺序是否合理。该评估体系超越了简单的二元判断，全面检验模型对漏洞路径的理解能力。

1.3.2 提出基于核心集的增量学习方法

针对模型演进过程中的灾难性遗忘问题，提出了一种基于参数变化特征的增量学习方法：

(1) 参数变化特征提取：对每个训练数据独立进行 LoRA 微调，计算训练前后模型参数的变化量作为该数据的特征表示。相比梯度特征，参数变化特征更能反映模型对数据的整体理解，且计算开销更低。

(2) 层次聚类核心集选择：对参数变化特征进行随机投影降维后，采用层次聚类算法将相似数据归为一类。通过从每个类中随机采样一个数据，构建代表性核心集。核心集大小可通过调整聚类数量灵活控制，实现数据多样性的有效表征。实验表明，仅使用 20% 的核心集即可达到与全量数据集相当的性能，训练时间减少约 45%。

(3) 混合相似度算法：综合考虑特征方向（余弦距离）和参数数值变化（欧式距离），提出混合相似度度量方法。实验表明，该方法在保持模型 AUC 接近全数据集训练结果的同时，Precision 和 Recall 指标显著优于单一距离度量方法。

1.3.3 设计基于数据精炼的质量提升方法

针对低质量数据集影响模型性能的问题，提出了一种基于模型行为分析的数据精炼方法：

(1) 有害数据识别：利用模型在验证集上的错误预测结果构建有害数据集，同时计算模型推理时的平均困惑度，将高困惑度数据纳入筛选范围。

(2) 聚类分布过滤：计算有害数据在训练集聚类结果中的分布，过滤掉与有害数据同属一个类别的训练数据，有效剔除潜在的低质量样本。

(3) 精炼数据集构建：将过滤后的训练数据与高困惑度数据重新组合，形成精炼数据集。该方法不仅适用于全量数据集，也可与核心集方法结合，在增量学习场景下实现数据质量的持续提升。

1.4 实验验证

为验证所提出方法的有效性，在多个数据集上进行了系统性实验。实验结果表明：(1) 基于大语言模型的漏洞检测方法在空指针异常检测任务上取得了优异的性能；(2) 任务形式设计是性能提升的关键因素，其效果显著优于传统的漏洞检测任务形式；(3) 基于核心集的增量学习方法能够用少量数据达到与全量数据集相当的性能，有效缓解灾难性遗忘问题；(4) 数据精炼方法能够显著提升模型在低质量数据集上的训练效果。详细的实验设计与结果分析将在第 6 章中系统阐述。

1.5 论文组织结构

论文共分为七章，各章内容安排如下：

第 1 章绪论：介绍研究背景与意义，分析国内外研究现状，阐述研究内容与主要贡献，概述实验验证情况。

第 2 章相关技术与研究现状：详细介绍软件漏洞检测技术、大语言模型基础、增量学习方法、数据精炼技术等理论基础，深入分析相关领域的研究进展。

第 3 章基于大语言模型的漏洞检测方法设计：阐述模型选择依据、QLoRA 微调技术、两级切片方法的设计原理与实现细节，以及多级评估体系的构建方法。

第 4 章基于核心集的增量学习方法：分析灾难性遗忘问题的成因，详细介绍参数变化特征提取、层次聚类核心集选择、混合相似度算法的技术原理与实现流程。

第 5 章基于数据精炼的质量提升方法：探讨低质量数据集对模型性能的影响机制，阐述有害数据识别、聚类分布过滤、精炼数据集构建的方法论。

第 6 章实验设计与结果分析：详细介绍实验环境、数据集、评估指标，系统呈现实验一（基础检测实验）、实验二（核心集实验）、实验三（数据精炼实验）的设计思路与结果分析。

第 7 章总结与展望：总结全文工作，分析研究局限，展望未来研究方向。

1.6 本章小结

作为论文的绪论，系统阐述了基于大语言模型的软件漏洞检测研究的背景与意义。首先分析了传统漏洞检测方法的局限性，介绍了大语言模型和 AI Agent 技术带来的新机遇，同时指出当前基于 Agent 的安全工具在资源消耗、时间效率、闭源依赖等方面存在的挑战。在此基础上，提出了技术思路：采用参数规模适中的开源大语言模型，结合轻量级静态分析工具，通过过程间切片构建覆盖完整缺陷传播路径的代码上下文，并要求模型输出结构化的缺陷路径信息，兼顾检测性能与可解释性。此外，还讨论了软件漏洞检测实际落地面临的两个关键问题——检测模型的持续演进能力与低质量数据集的性能影响，引出在增量学习与数据精炼方向的探索动机。然后从基于深度学习的漏洞检测、大语言模型在代码任务中的应用、增量学习与灾难性遗忘、数据质量与数据精炼四个方面梳理了国内外研究现状。在此基础上，详细介绍了三项主要贡献：设计基于大语言模型的漏洞检测方法、提出基于核心集的增量学习方法、设计基于数据精炼的质量提升方法。最后概述了实验验证工作，并给出了论文的组织结构。

第二章 相关技术与研究现状

2.1 引言

软件漏洞检测是软件安全领域的核心研究问题。随着软件系统规模和复杂度的持续增长，传统漏洞检测方法在应对新型漏洞和大规模代码分析时面临诸多挑战。近年来，基于深度学习的漏洞检测技术取得了显著进展，尤其是大语言模型的出现为漏洞检测提供了新的技术路径。

系统回顾与本研究相关的技术与研究工作。首先介绍软件漏洞检测技术的发展脉络，包括传统方法和基于深度学习的方法；然后讨论大语言模型在代码任务中的应用及其微调技术；接着分析增量学习与灾难性遗忘问题的研究现状；随后探讨数据质量对模型性能的影响及数据精炼技术；最后综述程序切片技术在漏洞检测中的应用。

2.2 软件漏洞检测技术

2.2.1 传统漏洞检测方法

传统漏洞检测方法主要包括静态分析、动态分析和人工代码审查三类。

静态分析（Static Analysis）通过预定义规则对源代码或二进制代码进行扫描，识别潜在的安全问题。代表性工具包括 Cppcheck^[1]、Clang Static Analyzer^[2]和 Infer^[3]。这些工具采用规则匹配、数据流分析等技术，能够在不执行程序的情况下快速检测漏洞。静态分析的优点是分析速度快、覆盖范围广，但存在误报率和漏报率较高的问题，且规则更新滞后于新漏洞类型的出现。

动态分析（Dynamic Analysis）通过实际执行或模拟执行程序来检测漏洞。模糊测试（Fuzzing）通过向程序输入大量随机测试数据，观察程序是否出现崩溃等异常行为来发现漏洞，代表性工具包括 AFL、libFuzzer 等。符号执行（Symbolic Execution）将程序输入符号化，通过路径约束求解探索程序执行路径，代表性工

具包括 KLEE、Angr 等。动态污点分析 (Dynamic Taint Analysis) 通过追踪污点数据在程序中的传播过程检测漏洞。动态分析能够发现真实可触发的漏洞, 但存在测试覆盖率有限、计算开销大等问题。

人工代码审查由安全专家对代码进行逐行检查, 理论上可以发现各种类型的漏洞, 但高度依赖审查者的专业知识与经验, 且在面对大规模代码库时效率极低。

2.2.2 基于深度学习的漏洞检测

基于深度学习的漏洞检测方法通过训练自动学习代码中的漏洞模式, 无需人工设计复杂的检测规则, 具有更强的适应性和泛化能力。

VulDeePecker^[4] 是最早将深度学习应用于漏洞检测的代表性工作之一。该方法将代码表示为数据依赖图 (Data Dependency Graph), 利用双向 LSTM 网络学习代码特征, 能够检测缓冲区溢出和缓冲区读取漏洞。Choi 等人^[6] 提出使用 Memory Network 从原始代码中端到端地预测缓冲区溢出, 无需手工设计特征。

Devign^[7] 从四个大型 C 语言开源项目 (Linux Kernel、QEMU、Wireshark、FFmpeg) 中收集并人工标注了函数级漏洞数据, 提出了基于门控图神经网络 (GGNN) 的漏洞检测方法, 在漏洞检测任务上取得了当时的最优性能。Fan 等人构建了 Big-Vul 数据集^[14], 从开源 GitHub 项目中收集了 3754 个 C/C++ 代码漏洞, 涵盖 91 种不同的漏洞类型。

然而, 现有数据集的质量问题逐渐引起研究关注。Jain 等人^[13] 对 C/C++ 漏洞检测数据集进行了系统性评估, 发现现有数据集普遍存在标注错误、漏洞类型单一、正负样本不平衡、上下文信息缺失等问题。这些问题直接影响了模型的训练效果和泛化能力。

Ding 等人^[15] 的研究进一步揭示, 现有漏洞检测基准测试严重高估了代码语言模型的性能。他们提出的 PrimeVul 数据集采用了严格的数据去重和时间顺序划分策略, 缓解了数据泄露问题。评估结果表明, 在 BigVul 上表现优异的模型在 PrimeVul 上性能急剧下降, 说明数据质量问题对模型评估的影响深远。

Risse 等人^[16] 指出, 当前机器学习漏洞检测研究的问题定义存在缺陷, 函数级二分类任务缺乏必要的上下文信息, 模型可能通过虚假相关性获得高分而非真正检测漏洞。这一发现对漏洞检测任务的评估方法提出了重要挑战。

Li 等人^[17] 系统研究了 LLM 在漏洞检测任务上的后训练管道，发现基于拒绝采样的监督微调（SFT）显著优于基于理性化的监督，且强化学习（RL）方法在偏好优化后能够进一步提升性能。Anthropic 发布的 Claude Code Security^[18] 通过扫描代码库检测安全漏洞并提供针对性修复建议，采用多阶段验证流程减少误报。

2.2.3 漏洞检测方法分类

表2-1对比了各类漏洞检测方法的特点。

表 2-1 漏洞检测方法对比

方法类别	分析速度	误报率	上下文支持	主要局限
静态分析	快	高 (30%-60%)	有限	规则维护成本高
动态分析	慢	低	完整	覆盖率有限
人工审查	极慢	极低	完整	效率低，依赖专家经验
深度学习 (小模型)	中等	中等	受限 (512 tokens)	复杂语义建模不足
大语言模型	中等	中等	完整 (8K-64K tokens)	需合理设计任务形式
本研究方法	较快	较低	完整	单一漏洞类型验证

2.3 大语言模型基础

2.3.1 Transformer 架构与代码预训练

Transformer 是 Vaswani 等人于 2017 年提出的一种基于自注意力机制（Self-Attention）的神经网络架构^[19]。Transformer 摒弃了传统的循环神经网络（RNN）和卷积神经网络（CNN）结构，完全基于注意力机制实现序列建模，具有并行化程度高、长距离依赖建模能力强等优点。

代码预训练与文本预训练有所不同，需要考虑代码的特殊性质：代码具有严格的语法规则；代码不仅有文本语义，还有执行语义，包括数据流、控制流等；代码中的变量定义和使用可能相隔很远，需要捕捉长距离依赖；实际项目通常包含多种编程语言，需要处理跨语言的交互。

StarCoder2^[8] 在 619 种编程语言的代码数据上进行预训练，显著提升了代码理解和生成能力。该模型使用 4TB 的代码数据进行训练，在多种代码基准测试上取得了优异成绩。

然而, Zhang 等人^[20]的研究表明, LLM 在代码生成任务上存在“熟悉度幻觉”(Specialist in Familiarity)问题, 即模型在当前数据集上的表现可能受到训练数据泄露的影响。他们提出了基于代码混淆的基准测试 OBFusEvAL, 通过多层次的混淆策略评估 LLM 在未见代码上的真实能力。研究结果表明, 经过混淆后, LLM 的测试通过率平均下降达 62.5%, 说明 LLM 在不熟悉代码任务上性能显著下降, 泛化能力有限, 需要持续演进以适应新的代码模式。

2.3.2 大语言模型的微调

预训练的大语言模型虽然学习到了丰富的知识, 但要应用于特定任务, 通常需要进行微调 (Fine-tuning)。微调是指在特定任务的标注数据上进一步训练模型, 使其适应目标任务。

全量微调 (Full Fine-tuning) 是指更新模型的所有参数, 通常能够取得最好的性能, 但存在计算开销大、存储开销大、灾难性遗忘和数据需求大等问题。

为了解决全量微调的问题, 研究者提出了参数高效微调 (Parameter-Efficient Fine-tuning, PEFT) 方法。PEFT 方法的核心思想是冻结预训练模型的大部分参数, 仅微调少量参数或引入少量可学习参数。Adapter^[21] 方法在 Transformer 层之间插入小型神经网络模块, 仅训练 Adapter 参数而冻结主干网络。Prefix Tuning^[22] 方法在输入序列前添加可学习的连续前缀向量, 引导模型生成特定任务的输出。

LoRA (Low-Rank Adaptation)^[23] 方法通过低秩分解近似参数更新, 将参数更新表示为两个低秩矩阵的乘积。对于 GPT-3, LoRA 可以将 trainable 参数减少 10000 倍, 计算硬件需求减少 3 倍, 同时保持与全量微调相当或更好的模型质量。

QLoRA (Quantized LoRA)^[24] 是 Dettmers 等人于 2023 年提出的高效微调方法, 结合了量化技术和 LoRA 微调。QLoRA 的核心技术包括: (1) 4 比特量化: 将预训练模型权重量化为 4 比特, 大幅降低显存占用。QLoRA 提出了一种新的 4 比特数据类型 NF4 (NormalFloat 4), 专门针对正态分布的权重进行了优化。(2) 双重量化: 为了进一步减少显存占用, QLoRA 对量化常数也进行了量化, 使用 8 比特量化来表示 32 比特的量化常数。(3) 分页优化器: QLoRA 使用分页优化器 (Paged Optimizer) 来管理显存, 当显存不足时将优化器状态暂时换出到 CPU 内存, 需要时再换回 GPU。QLoRA 能够在单张 48GB GPU 上微调 65B 参数模型, 同时保持完整的 16 比特微调任务性能。

2.4 增量学习与灾难性遗忘

2.4.1 灾难性遗忘问题

灾难性遗忘（Catastrophic Forgetting）是神经网络在持续学习过程中面临的普遍问题，最早由 McCloskey 和 Cohen 于 1989 年发现。Kirkpatrick 等人^[25]将这一问题形式化为克服神经网络中的灾难性遗忘，并提出了 EWC（Elastic Weight Consolidation）方法。当神经网络学习新任务或新数据时，参数更新会覆盖之前学习到的知识，导致模型在旧任务上的性能显著下降。

Toneva 等人^[9]通过实证研究发现，深度神经网络在学习新数据时会遗忘之前学习过的样本。他们提出了“遗忘事件”（Forgetting Event）的概念，即模型对某个样本从正确预测变为错误预测的时刻。研究发现，遗忘事件在训练过程中频繁发生，且与样本的难度和代表性有关。基于遗忘动态，可以从训练数据中省略相当一部分样本，同时保持最先进的泛化性能。

在大语言模型场景下，灾难性遗忘问题更加突出。Chen 等人^[26]系统比较了监督微调（SFT）和强化学习（RL）两种后训练方法的遗忘模式。实验结果表明，RL 比 SFT 导致的遗忘更少，同时能够达到相当或更高的目标任务性能。这归因于 RL 使用 on-policy 数据的 mode-seeking 特性，能够在学习目标任务时保持先验知识完整。

2.4.2 增量学习方法分类

Wang 等人^[10]对持续学习进行了全面综述，将现有方法分为三大类：基于回放、基于正则化和基于参数隔离。表2-2对比了各类增量学习方法的特点。

表 2-2 增量学习方法对比

方法类别	存储开销	计算开销	遗忘率	代表性方法
基于回放	中等 (需存储样本)	低	低	GSS, GCR, OCS
基于正则化	低 (仅存储重要性)	中等	中等	EWC, GEM
基于参数隔离	高 (参数随任务增长)	高	极低	PNN, DEN
本研究方法	低 (核心集 10%)	低	低	参数变化特征 + 层次聚类

基于回放的方法

基于回放 (Replay-based) 的方法通过保留一部分旧数据或使用生成模型合成旧数据的代表样本, 在训练新任务时同时训练旧数据, 从而缓解遗忘问题。

经验回放 (Experience Replay) 是最简单的回放方法, 维护一个固定大小的回放缓冲区, 随机存储旧任务的部分样本。训练新任务时, 从缓冲区采样旧样本混合训练。生成回放 (Generative Replay) 使用生成模型 (如 GAN、VAE) 学习旧任务的分布, 生成合成样本用于回放, 不需要存储真实样本, 但生成质量影响回放效果。

核心集选择 (Coreset Selection) 方法旨在从旧数据中选择少量代表性样本, 构建核心集用于回放。Aljundi 等人^[11] 提出了基于梯度的样本选择方法 (GSS), 通过计算样本梯度之间的余弦相似度来选择代表性样本。Yoon 等人^[27] 提出了在线核心集选择 (OCS) 方法, 在每次迭代中选择最具代表性和信息性的核心集, 以在线方式训练。Tiwari 等人^[12] 提出了梯度核心集回放 (GCR) 方法, 将核心集选择问题转化为优化问题, 选择 closely approximates 当前所有数据梯度的核心集。

基于正则化的方法

基于正则化 (Regularization-based) 的方法通过在损失函数中添加正则化项, 约束参数更新不偏离旧任务的最优解太远。

EWC (Elastic Weight Consolidation)^[25] 计算每个参数对旧任务的重要性, 对重要参数的更新施加更大的惩罚。EWC 使用 Fisher 信息矩阵近似参数重要性, 能够有效保护旧任务的关键参数。GEM (Gradient Episodic Memory)^[28] 利用梯度方向约束新任务的学习不干扰旧任务, 通过存储旧任务的梯度信息来指导新任务的学习。

基于参数隔离的方法

基于参数隔离 (Parameter Isolation-based) 的方法为不同任务分配不同的参数子集, 避免任务之间的干扰。

PNN (Progressive Neural Network) 为每个新任务分配新的网络列, 通过横

向连接复用旧任务的知识，完全避免了遗忘，但参数数量随任务数量线性增长。DEN (Dynamically Expandable Network) 动态扩展网络结构，根据任务复杂度决定是否新增神经元。

2.4.3 大语言模型的增量学习

大语言模型的增量学习面临新的挑战：参数量巨大，存储每个样本的梯度变化计算开销和存储开销都非常高；任务复杂性，大语言模型通常处理多任务，任务边界不清晰；计算资源限制，全量微调大语言模型需要大量计算资源。

Wu 等人^[29] 对大语言模型的持续学习进行了系统调研，将持续学习技术分为持续预训练、指令微调和 alignment 三个阶段进行 categorization。Wang 等人^[30] 提出了 TRACE 基准，用于评估大语言模型的持续学习能力。实验结果表明，经过 TRACE 训练后，对齐的大语言模型在通用能力和指令跟随能力上均出现显著下降。

Song 等人^[31] 提出了持续参数高效微调 (ConPET) 方法，将参数高效微调应用于持续学习场景。ConPET 包括静态和动态两个版本，静态版本通过 PET 和动态回放策略将可训练参数规模减少 3000 多倍，动态版本采用独立的 PET 模块和选择器实现可扩展的持续学习。Cheng 等人^[32] 探索了领域持续预训练对大语言模型的影响，发现基于阅读理解的训练方法能够显著提升领域性能，同时保持通用能力。

在漏洞检测领域，Xue 等人^[33] 提出了 VulTypeIL 方法，采用混合回放策略和 EWC 正则化缓解灾难性遗忘，结合 focal loss 和 label smooth 处理长尾分布问题，在软件漏洞类型识别任务上取得了优于现有方法的效果。

2.5 数据质量与数据精炼

2.5.1 数据质量对模型性能的影响

数据质量对深度学习模型的性能有着决定性影响。Jain 等人^[13] 对 C/C++ 漏洞检测数据集进行了系统性评估，发现现有数据集普遍存在标注错误、漏洞类型单一、正负样本不平衡、上下文信息缺失和数据泄露等问题。这些问题直接影响了模型的训练效果和泛化能力。

Ding 等人^[15]的研究进一步揭示，现有漏洞检测基准测试严重高估了代码语言模型的性能。他们提出的 PrimeVul 数据集采用了严格的数据去重和时间顺序划分策略，缓解了数据泄露问题。评估结果表明，在 BigVul 上表现优异的模型在 PrimeVul 上性能急剧下降，说明数据质量问题对模型评估的影响深远。

2.5.2 数据精炼技术

数据精炼（Data Refinement）是提升数据质量的重要手段，主要包括数据清洗、数据增强、困难样本挖掘等方法。

数据清洗是指识别并纠正或删除数据中的错误、噪声和不一致。规则过滤基于预定义规则过滤明显错误的数据，如代码无法编译、长度异常等。模型辅助清洗利用预训练模型检测异常样本，如使用困惑度识别标注错误。人工审核对可疑样本进行人工检查和修正，确保数据质量。

困难样本挖掘（Hard Example Mining）是指识别模型难以正确预测的样本，重点训练这些样本以提升模型性能。困惑度筛选计算模型对样本的困惑度，高困惑度样本通常是困难样本。损失筛选选择训练损失高的样本进行重点训练。错误样本分析模型在验证集上的错误预测，构建困难样本集。

Xia 等人^[34]提出了 LESS 方法，用于指令调优中的影响数据选择。LESS 通过低阶梯度相似性搜索（Low-rank gradiEnt Similarity Search）估计数据影响度，实验表明在 5% 的数据上训练往往能够优于在全量数据上训练的效果。

Wang 等人^[35]提出了 In-Run Data Shapley 方法，在单次训练过程中计算数据贡献度，实现可扩展的数据归因。该方法通过梯度信息高效地评估每个训练样本对最终模型性能的贡献，为数据精炼提供了理论基础。

2.6 程序切片技术

2.6.1 程序切片的基本概念

程序切片（Program Slicing）是由 Weiser 于 1981 年提出的一种程序分析技术，旨在从程序中提取与特定变量或操作相关的代码片段。程序切片通过分析程序的控制流和数据流，确定对目标变量有影响的语句集合。

形式化地, 给定程序 P 、切片标准 $C = (q, V)$ (其中 q 是程序中的某个点, V 是变量集合), 程序切片是 P 的一个子集 P' , 满足 P' 在 C 上的行为与 P 相同。

前向切片 (Forward Slicing) 从切片标准出发, 找出所有受其影响的语句, 用于分析变量的影响范围。后向切片 (Backward Slicing) 从切片标准出发, 找出所有影响它的语句, 用于分析变量的依赖来源。静态切片 (Static Slicing) 不考虑具体输入, 分析所有可能执行路径上的依赖关系, 保守但可能包含冗余。动态切片 (Dynamic Slicing) 基于具体输入和执行轨迹, 分析实际执行路径上的依赖关系, 精确但依赖于具体输入。

2.6.2 程序依赖图

程序依赖图 (Program Dependence Graph, PDG) 是程序切片的重要基础, 由 Ferrante 等人于 1987 年提出^[36]。PDG 将程序表示为图结构, 节点表示语句, 边表示依赖关系。

控制依赖 (Control Dependence) 描述语句之间的控制关系。如果语句 A 的执行决定是否影响语句 B 的执行, 则称 B 控制依赖于 A 。控制依赖通常由条件语句 (如 if、while、for) 产生。

数据依赖 (Data Dependence) 描述语句之间的数据流关系。如果语句 A 定义的变量被语句 B 使用, 且 A 和 B 之间存在可达的执行路径, 则称 B 数据依赖于 A 。

2.6.3 切片在漏洞检测中的应用

程序切片在漏洞检测中具有重要应用价值。Lu 等人^[37] 在工业级 C++ 代码库上提出了代码切片算法用于上下文提取, 结合多角色 LLM 框架实现缺陷导向的自动化代码审查, 在关键 bug 包含率 (KBI) 上取得了 2 倍于标准 LLM 的提升。

Guo 等人^[38] 提出了 RepoAudit, 一个自主的 LLM Agent 用于仓库级代码审计。该框架通过分析可行程序路径上的数据流事实来探索代码库, 采用验证器模块减轻幻觉, 在 15 个真实世界项目中检测到 40 个真实 bug, 精确率达 78.43%。

Yu 等人^[39] 提出了 OrcaLoca 框架, 通过优先级调度、动作分解与相关性评分、

距离感知上下文剪枝等策略,在 SWE-bench Lite 上达到了 65.33% 的函数匹配率,成为新的开源 SOTA。Sohrabizadeh 等人^[40]提出了 Nemotron-CORTEXA,通过代码嵌入模型和定位 Agent 实现精细化检索,在 SWE-bench 上取得了比 Agentless 更高的问题解决率和更低的成本。

Bingham 等人^[41]提出了随机投影方法用于降维,该技术在处理高维代码特征时可以显著降低计算开销,同时保持数据向量之间的相似性。

两级切片方法结合了基于调用图的粗粒度切片和基于 PDG 的细粒度切片,在计算开销和切片质量之间取得平衡,为漏洞检测提供高质量的输入。

2.7 本章小结

系统回顾了与基于大语言模型的软件漏洞检测相关的技术与研究工作。在传统漏洞检测方法方面,静态分析、动态分析和人工审查各有优劣,但均面临扩展性和效率问题。基于深度学习的方法通过自动学习代码特征,在漏洞检测任务上展现出巨大潜力,但现有数据集质量问题和评估方法缺陷仍是主要挑战。

大语言模型为漏洞检测提供了新的技术路径,但其参数量巨大导致全量微调成本高昂,参数高效微调技术如 LoRA 和 QLoRA 显著降低了微调门槛。增量学习中的灾难性遗忘问题在大语言模型场景下更加突出,基于回放、正则化和参数隔离的方法各有特点,但如何在大模型场景下高效应用仍是开放问题。

数据质量对模型性能具有决定性影响,现有漏洞检测数据集普遍存在标注错误、数据泄露等问题。数据精炼技术通过清洗、增强和困难样本挖掘提升数据质量,有害数据过滤能够有效剔除对模型训练产生负面影响的数据。程序切片技术通过提取与漏洞相关的代码片段,为漏洞检测提供高质量的上下文输入,在工业级代码审计中展现出良好应用前景。

现有研究为基于大语言模型的软件漏洞检测提供了坚实的理论基础和技术支撑,但在大语言模型漏洞检测的持续演进、数据质量提升和高效上下文构建等方面仍存在研究空间,这也是本研究的出发点。

第三章 基于大语言模型的漏洞检测方法设计

3.1 引言

软件漏洞检测是软件安全领域的重要研究课题。传统的漏洞检测方法如静态分析工具、动态分析工具和人工代码审查等，虽然在一定程度上能够发现软件中的安全问题，但随着软件复杂性的增加和攻击技术的不断演进，这些方法逐渐显现出局限性。基于规则的方法需要专家手动编写和维护检测规则，难以应对新出现的漏洞类型；动态分析方法计算开销大，难以处理大规模程序；人工代码审查效率低，无法满足现代软件开发快速迭代的需求。

针对上述挑战，提出了一种基于大语言模型的漏洞检测方法。核心思路在于：充分利用静态分析工具在上下文构建与上下文限定方面的精确性，通过过程间切片技术构建能够覆盖完整缺陷传播路径的代码上下文，使模型输入尽可能涵盖与缺陷分析相关的全部语义信息；同时，设计结构化的输出格式，要求模型明确给出从缺陷触发点（Source）到缺陷影响点（Sink）的完整数据流路径，而非仅输出二元判定结果。这种设计既使模型能够在完整的缺陷上下文中进行推理，提升了检测的准确性和可解释性，又使输出结果能够直接被开发者理解和用于漏洞修复，降低了人工验证成本。

3.2 整体方法框架

基于大语言模型的漏洞检测方法的整体框架如图3-1所示。该方法以源代码文件作为输入，经过代码解析与上下文构建处理后形成模型可处理的输入序列，通过微调后的大语言模型进行推理，输出漏洞检测结果和缺陷路径。

方法的核心流程包括以下步骤：

(1) **代码解析与上下文构建**：对待检测的源代码文件进行解析，构建函数调用图。基于静态分析工具识别的 Source 语句，采用过程间切片技术提取相关的代码片段。上下文构建过程遵循可靠（sound）性原则，确保不遗漏与缺陷分析

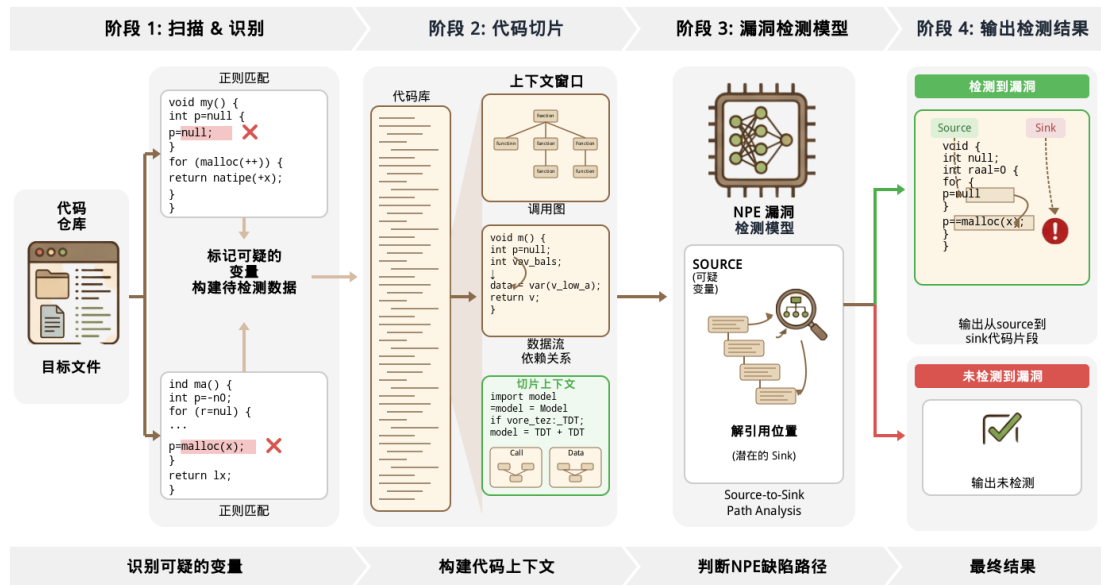


图 3-1 基于大语言模型的漏洞检测方法框架图

相关的任何代码。这一阶段的核心目标是将传统仅关注单个函数内部的过程内分析，扩展到跨函数的过程间分析，从而捕获完整的缺陷传播路径。

(2) **模型推理**：将构建后的上下文输入到微调后的大语言模型中。模型基于学习到的漏洞知识，判断代码中是否存在漏洞，并预测从 Source 到 Sink 的缺陷路径。实验表明，上下文的完整性（是否覆盖跨函数的缺陷传播路径）是影响检测性能的关键因素，而具体的任务形式（是否要求输出路径）对性能影响较小。

(3) **结果评估**：采用三级评估体系对模型预测结果进行评估。一级评估检测模型是否正确预测缺陷存在及 Source 位置；二级评估检查预测的 Sink 语句是否与 Source 存在依赖关系；三级评估验证从 Source 到 Sink 的依赖传递顺序是否合理。

3.3 模型选择与优化

3.3.1 基座模型选择

在模型选择阶段，需要综合考虑检测性能、硬件资源限制和部署可行性等多方面因素。小模型通常需要将代码转化为控制流图、程序依赖图等中间表示形式，增加了计算开销和误差引入，且输入长度受限较大（如 CodeBERT 为 512 个 Token），难以处理长代码片段。相比之下，大语言模型可以直接处理原始代

码序列，支持较长的输入序列，且在海量代码语料上进行了预训练，已经学习到了丰富的代码相关知识，微调效率更高。

基于上述分析，选择大语言模型作为基础模型。现有预训练大语言模型在代码任务上的表现受到广泛关注，特别需要关注不同模型在处理 C/C++ 代码任务时的能力。候选模型包括 CodeLlama-7B、DeepSeek-Coder-33B、CodeQwen2.5-7B、StarCoder-7B 等。根据评估结果，CodeQwen2.5-7B-chat 在处理 C/C++ 代码任务时表现优异，考虑到实际部署中与合作项目组的硬件资源限制，最终选择 CodeQwen2.5-7B-chat 作为基座模型。选择理由如下：

- (1) **性能优异**：CodeQwen2.5-7B 在 C/C++ 代码理解任务上的表现与更大参数的模型相当；
- (2) **参数量适中**：70 亿参数量在单卡 H800 上可以运行，便于部署和推理；
- (3) **上下文长度支持好**：CodeQwen2.5 支持 64K 上下文长度，模型本身对长序列的处理能力较强；
- (4) **开源许可友好**：CodeQwen2.5 采用 Apache 2.0 开源协议，允许商业使用。

3.3.2 QLoRA 微调技术

大语言模型的全量微调需要巨大的显存和计算资源，且容易导致模型过拟合和丧失预训练知识的泛化能力。为了避免这些问题，采用 QLoRA（Quantized Low-Rank Adaptation）微调技术。

LoRA 的核心思想是通过低秩分解近似参数更新。对于预训练模型中的每个权重矩阵 $W \in \mathbb{R}^{d \times k}$ ，LoRA 将其更新表示为低秩矩阵的乘积：

$$W' = W + \Delta W = W + BA \quad (3-1)$$

其中， $B \in \mathbb{R}^{d \times r}$ ， $A \in \mathbb{R}^{r \times k}$ ， $r \ll \min(d, k)$ 是秩超参数。训练过程中冻结预训练权重 W ，仅优化 A 和 B 。

QLoRA 在 LoRA 的基础上引入了 4 比特量化技术，进一步降低了显存占用。QLoRA 提出了一种新的 4 比特 NormalFloat 量化格式 NF4，专门针对正态分布的权重进行了优化，并采用双重量化技术对量化常数也进行量化，使用 8 比特量化来表示 32 比特的量化常数。此外，QLoRA 使用分页优化器管理显存，避免

峰值显存溢出。

3.4 上下文构建方法

在大语言模型应用于代码缺陷检测任务时，输入上下文的质量直接影响模型的推理效果。由于软件项目通常包含大规模代码库，而大语言模型的上下文窗口存在物理限制，如何构建既完整又精简的输入上下文是面临的关键挑战。

从上下文工程（Context Engineering）的视角出发，设计了基于静态分析的上下文构建方法。该方法的核心目标是：在有限的上下文窗口内，最大化保留与缺陷分析相关的语义信息，同时剔除无关代码的干扰。上下文构建过程遵循可靠性（Soundness）原则，确保不遗漏与缺陷传播路径相关的任何关键代码。

3.4.1 Source 锚点识别

上下文构建的起点是识别缺陷分析的锚点，即 Source 语句。Source 语句是指触发一个缺陷所需执行的代码片段的起始位置，通常是变量定义、函数调用、内存分配等操作。Source 语句由静态分析工具自动识别，作为后续上下文提取的参考点。

例如，在空指针解引用缺陷中，可能导致空指针的变量赋值语句即为 Source 语句；在缓冲区溢出缺陷中，未进行边界检查的数组访问语句即为 Source 语句。

3.4.2 函数级上下文提取

在识别 Source 语句后，需要确定与之相关的函数集合。软件缺陷往往不是局限于单个函数内部，而是沿着函数调用链传播。因此，上下文构建需要跨越函数边界，捕捉完整的缺陷传播路径。

基于调用图（Call Graph）的函数级上下文提取方法如下：从包含 Source 语句的函数出发，沿调用边向上追溯调用该函数的函数集合（调用者），沿被调用边向下追踪被该函数调用的函数集合（被调用者）。追溯深度根据缺陷类型和代码复杂度动态调整，通常设置为 3-5 层即可覆盖绝大多数缺陷传播路径。

函数级上下文提取在计算开销和覆盖范围之间取得平衡，能够快速确定与缺陷分析相关的函数范围，为后续的语句级筛选奠定基础。

3.4.3 语句级上下文筛选

函数级上下文提取可能包含大量与缺陷分析无关的代码。为进一步提升上下文的信噪比，需要在每个函数内部进行语句级筛选，仅保留与 Source 变量存在依赖关系的语句。

基于程序依赖图 (Program Dependence Graph, PDG) 的语句级上下文筛选方法如下：首先构建函数内部的程序依赖图，识别数据依赖和控制依赖关系；然后从 Source 语句出发，沿依赖边进行双向遍历——后向遍历收集所有可能影响 Source 语句执行的语句（依赖来源），前向遍历收集所有可能被 Source 语句影响的语句（影响目标）；最后合并后向和前向遍历的结果，形成语句级上下文。

语句级上下文筛选能够有效剔除函数内部的无关代码，显著减少输入 Token 数量，降低模型推理成本，同时保持缺陷传播路径的完整性。

3.4.4 上下文组合策略

函数级上下文提取和语句级上下文筛选可以灵活组合使用，以适应不同的场景需求：

组合模式：先使用函数级上下文提取确定涉及的函数集合，再使用语句级上下文筛选在每个函数内部进行精细筛选。这种组合策略在保持上下文可靠性的同时，有效减少了输入 Token 数量，是推荐的标准配置。

独立模式：当计算资源有限或需要快速处理大量代码时，可以仅使用函数级上下文提取；当需要高精度的上下文结果或处理复杂缺陷时，可以仅使用语句级上下文筛选。

通过上述上下文构建方法，能够在有限的上下文窗口内，为模型提供包含完整缺陷传播路径信息的高质量输入，为后续的模式推理奠定基础。需要强调的是，上下文构建的核心价值在于实现从过程内分析到过程间分析的跃迁。实验表明，正是这种分析范围的扩展——而非任务形式的具体设计——是检测性能提升的根本来源。

3.5 多级评估体系

3.5.1 评估需求分析

传统的漏洞检测评估方法通常仅关注模型是否能够正确识别漏洞的存在（即二分类的准确率、精确率、召回率等指标）。然而，对于基于大语言模型的漏洞检测方法，这种评估方式存在以下不足：首先，忽略路径信息，无法评估模型对漏洞传播过程的理解；其次，可能误判正确回复，模型可能猜对漏洞类型但给出的路径完全错误；最后，不利于后续优化，无法定位模型的具体薄弱环节。

针对上述问题，引入了一种新的多级评估框架，从缺陷检测到路径预测，逐步深入地检验模型的能力。

3.5.2 三级评估指标

定义 3.3（一级评估）：一级评估的目标是评估模型是否能够准确预测程序是否有缺陷，并且模型预测的源点（Source）语句与提供给模型的源点语句完全一致。

形式化地，对于测试样本 (x, y) ，其中 x 是输入代码上下文， $y = (l, s_{src}, p)$ 是标签， l 表示缺陷是否存在， s_{src} 表示 Source 语句， p 表示缺陷路径。模型的预测输出为 $\hat{y} = (\hat{l}, \hat{s}_{src}, \hat{p})$ 。一级评估通过的条件是：

$$\text{Level-1 Pass} \iff (\hat{l} = l) \wedge (\hat{s}_{src} = s_{src}) \quad (3-2)$$

一级评估是最基本的要求，确保了模型能够正确地识别出潜在的问题所在。

定义 3.4（二级评估）：在满足一级评估标准的前提下，进一步检查模型预测的终点（Sink）语句是否与 Source 语句之间存在依赖关系。

形式化地，二级评估通过的条件是：

$$\text{Level-2 Pass} \iff \text{Level-1 Pass} \wedge \text{Dependent}(\hat{s}_{sink}, s_{src}) \quad (3-3)$$

其中， $\text{Dependent}(s_1, s_2)$ 表示语句 s_1 和 s_2 之间存在数据依赖或控制依赖关系，可通过查询程序依赖图获得。

定义 3.5 (三级评估): 在前两级评估均通过的情况下, 进一步验证从 Source 语句到 Sink 语句的过程中, Source 语句依赖的传递顺序是否与模型预测路径中的依赖传递顺序一致。

形式化地, 设标准答案的缺陷路径为 $p = (s_0, s_1, \dots, s_n)$, 模型预测的路径为 $\hat{p} = (\hat{s}_0, \hat{s}_1, \dots, \hat{s}_m)$ 。三级评估通过的条件是:

$$\text{Level-3 Pass} \iff \text{Level-2 Pass} \wedge \text{OrderMatch}(p, \hat{p}) \quad (3-4)$$

其中, OrderMatch 函数检查两条路径中依赖传递顺序是否一致。

3.5.3 评估流程与指标计算

基于上述三级评估指标, 评估流程如图3-2所示。

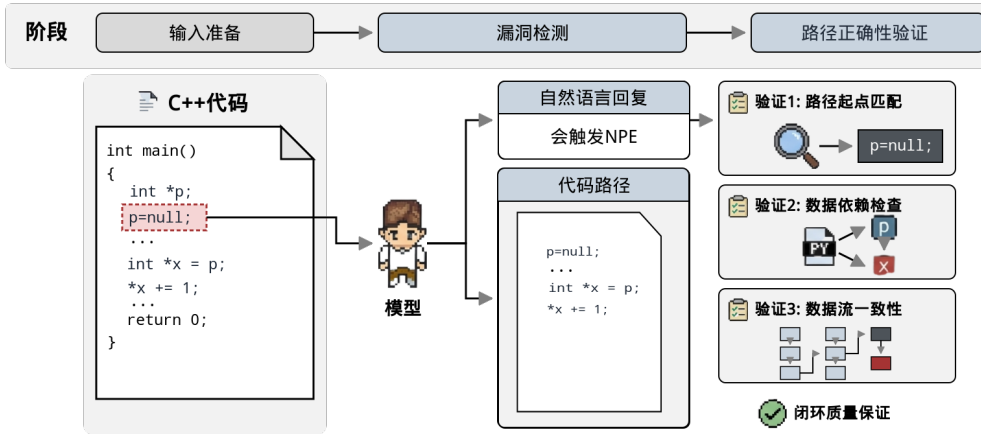


图 3-2 三阶段评估流程图

评估流程如下: 首先准备测试数据集, 确保每个样本包含代码上下文、缺陷标签、Source 语句和标准缺陷路径; 然后使用训练好的模型对测试集进行推理, 获取模型的预测结果; 依次进行一级、二级、三级评估, 记录各级评估通过的样本数量; 最后计算各级评估的通过率。

设 N 为测试样本总数, N_1 、 N_2 、 N_3 分别为通过一级、二级、三级评估的样本数量, 则各级评估的通过率为:

$$P_1 = N_1/N, \quad P_2 = N_2/N, \quad P_3 = N_3/N \quad (3-5)$$

3.5.4 评估方法的优势

提出的多级评估体系具有以下优势：

(1) 无需预知 Sink 即可评估路径正确性：这是多级评估的核心优势。传统评估方法需要预先标注每个 Source 对应的 Sink 语句，但在实际场景中，缺陷路径可能有多条，人工标注难以穷尽所有合理路径。多级评估通过依赖关系检查而非精确匹配来判断模型输出的正确性：二级评估仅要求模型预测的 Sink 与 Source 存在依赖关系，三级评估仅要求依赖传递顺序合理，而不要求与标准答案完全一致。这种设计显著降低了标注成本，同时能够容纳多条合理的缺陷路径。

(2) 便于人工快速确认：虽然基于依赖关系的评估不如精确匹配严格，但其输出结果具有清晰的结构化形式——模型需要明确给出从 Source 到 Sink 的完整路径。人工审核人员只需检查路径中各语句之间是否存在合理的数据依赖或控制依赖关系，即可快速判断模型输出是否正确，无需深入理解完整的代码语义。

(3) 可诊断性：三级评估形成递进式的诊断链条。若模型在一级评估失败，说明其无法正确识别缺陷存在或定位 Source；若通过一级但失败于二级，说明模型理解缺陷类型但未能正确识别影响范围；若通过前两级但失败于三级，说明模型对缺陷传播顺序的理解存在偏差。这种细粒度的诊断信息有助于针对性地优化模型。

(4) 低成本：依赖关系信息可以直接从程序依赖图中自动获取，无需人工额外标注，评估过程完全自动化。

3.6 本章小结

详细介绍了基于大语言模型的漏洞检测方法的设计与实现。首先阐述了整体方法框架，包括代码解析与上下文构建、模型推理、结果评估三个核心环节。在模型选择与优化方面，选择 CodeQwen2.5-7B-chat 作为基座模型，并采用 QLoRA 技术进行高效微调，在单卡 H800 上支持较长的上下文窗口。在上下文构建方面，采用过程间切片技术，包括函数级上下文提取和语句级上下文筛选，遵循可靠性原则确保不遗漏与缺陷分析相关的任何代码。通过静态分析工具的精确切片，将能够覆盖完整缺陷传播路径的代码段作为模型输入，使大模型能够在更完整的语义上下文中进行推理，这是检测性能提升的关键所在。在评估体系方面，提

出了三级评估指标，其核心优势在于无需预知 **Source** 对应的 **Sink** 即可通过依赖关系检查有效评估路径的正确性，虽然不如精确匹配严格，但显著降低了标注成本且便于人工快速确认。

第四章 基于核心集的增量学习方法

4.1 引言

随着软件系统的不断发展，新的编程语言语法更新和新的缺陷模式不断出现，已有的漏洞检测模型需要不断地学习新的知识以保持其有效性。然而，直接使用新的数据更新模型可能导致灾难性遗忘（Catastrophic Forgetting）问题，即模型在学习新知识时会遗忘旧知识，导致原有检测能力显著下降。

为了解决这一问题，提出了一种基于核心集的增量学习方法。该方法的核心思想是：从旧训练集中选取一部分代表性样本构建核心集，并在更新模型时使用核心集与新数据集进行混合训练，从而在保持新任务性能的同时，有效缓解对旧知识的遗忘。方法的整体流程如图4-1所示。

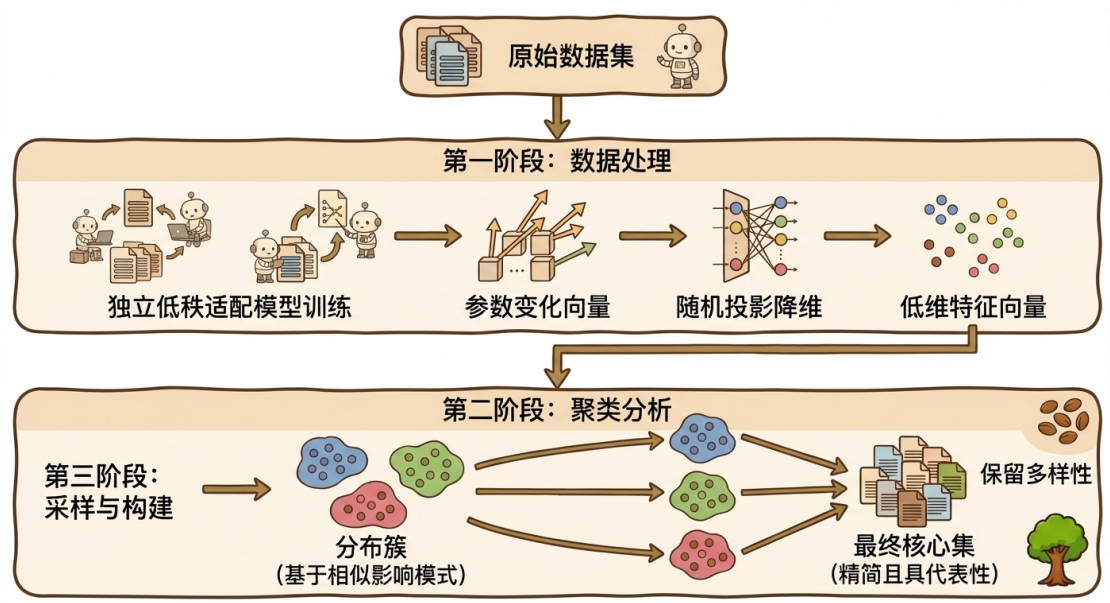


图 4-1 基于核心集的增量学习方法整体框架

主要贡献包括：（1）提出参数变化特征表示方法，利用训练前后模型参数的变化量表征数据对模型的影响；（2）设计层次聚类核心集选择算法，确保核心集能够有效覆盖原始数据的分布；（3）提出混合相似度算法，综合考虑特征方向和幅度信息，提升核心集的代表性。

4.2 灾难性遗忘问题分析

4.2.1 遗忘现象的观察

为直观理解灾难性遗忘现象，进行了观察实验：使用缺陷类型 A 的数据集训练模型 M_1 ，然后使用缺陷类型 B 的数据集在 M_1 的基础上继续训练得到模型 M_2 。实验结果显示，模型 M_2 在缺陷类型 A 测试集上的性能显著下降，而缺陷类型 B 测试集上的性能则明显提升。这一现象表明，模型在学习新的缺陷检测任务时，确实会遗忘之前学到的知识，导致对已学习缺陷类型的检测能力退化。

4.2.2 理论分析

从优化角度来看，灾难性遗忘的根本原因在于神经网络的参数共享特性。设模型参数为 θ ，任务 A 的损失函数为 $L_A(\theta)$ 。当模型学习任务 A 时，参数更新到 $\theta_A = \arg \min_{\theta} L_A(\theta)$ 。当模型接着学习任务 B 时，参数更新到 $\theta_B = \arg \min_{\theta} L_B(\theta)$ 。如果任务 A 和任务 B 的损失函数梯度方向不一致，即 $\nabla_{\theta} L_A(\theta) \cdot \nabla_{\theta} L_B(\theta) \neq \|\nabla_{\theta} L_A(\theta)\| \|\nabla_{\theta} L_B(\theta)\|$ ，则学习任务 B 的参数更新会破坏任务 A 的最优解，导致模型在任务 A 上的性能下降。

4.2.3 增量学习的挑战

理想的增量学习方法应该满足以下要求：低遗忘，学习新知识后对旧知识的遗忘程度低；高效率，计算和存储开销可控；可扩展，能够适应新任务的到来；无需原始数据，不需要存储全部旧数据。然而，现有的增量学习方法在应用于大语言模型时面临新的挑战：大语言模型参数量巨大，存储每个样本的梯度或参数变化计算开销和存储开销都非常高；漏洞检测任务边界模糊，不同缺陷类型之间可能存在重叠；计算资源有限，难以支持复杂的增量学习算法。

4.3 方法概述

提出的基于核心集的增量学习方法主要包括三个阶段：参数变化特征提取、层次聚类核心集选择、模型更新与回放训练。

参数变化特征提取阶段对旧训练集中的每一个数据独立进行 LoRA 微调, 记录训练前后 LoRA 参数的变化量作为该数据的特征表示。参数变化特征反映了数据对模型的整体影响, 相比单步梯度特征更加鲁棒, 且可以复用训练过程, 无额外计算开销。层次聚类核心集选择阶段对参数变化特征进行随机投影降维后, 采用层次聚类算法将相似的数据归为一类, 通过从每个类中随机采样一个样本构建代表性核心集。模型更新与回放训练阶段将核心集与新数据集混合进行训练, 通过回放核心集样本, 使模型能够在学习新知识的同时保持对旧知识的记忆。

算法1展示了基于核心集的增量学习整体流程。

Algorithm 1 基于核心集的增量学习算法

```

1: Input: 旧数据集  $D_{old}$ , 新数据集  $D_{new}$ , 核心集比例  $\alpha$ 
2: Output: 更新后的模型  $M_{updated}$ 
3: // 阶段 1: 参数变化特征提取
4: for 每个样本  $d_i \in D_{old}$  do
5:   使用  $d_i$  对基座模型进行 LoRA 微调, 得到参数  $\phi_i^*$ 
6:   计算参数变化  $\Delta\phi_i = \phi_i^* - \phi_{base}$ 
7:   展平为一维向量  $v_i = \text{flatten}(\Delta\phi_i)$ 
8:   随机投影降维  $u_i = \frac{1}{\sqrt{k}} R v_i$ , 其中  $R \in \mathbb{R}^{k \times M}$ 
9: end for
10: // 阶段 2: 层次聚类核心集选择
11: 对特征向量  $\{u_1, u_2, \dots, u_N\}$  进行层次聚类, 得到  $K = \alpha \cdot N$  个类
12: for 每个类  $C_j$  do
13:   随机采样一个样本  $d_j^* \in C_j$ 
14: end for
15: 构建核心集  $D_{coreset} = \{d_1^*, d_2^*, \dots, d_K^*\}$ 
16: // 阶段 3: 模型更新与回放训练
17: 混合数据集  $D_{mix} = D_{coreset} \cup D_{new}$ 
18: 在  $D_{mix}$  上训练模型, 得到  $M_{updated}$ 
19: return  $M_{updated}$ 

```

4.4 参数变化特征提取

4.4.1 核心洞察

方法基于以下核心洞察: 模型参数的变化量可以有效表征数据对模型的影响。具体而言, 不同性质的训练数据会引导模型参数产生具有区分性的变化模式, 这些变化模式蕴含着数据与模型交互的关键信息。

首先,不同缺陷类型的样本会导致参数产生不同的变化方向。例如,缓冲区溢出漏洞样本和资源泄漏漏洞样本由于具有不同的代码特征和漏洞模式,会引导参数向量向不同的方向更新。其次,具有相似漏洞模式的样本会产生相似的参数变化,表现为参数变化向量之间具有较小的夹角。最后,模型的困难样本通常会导致更大幅度的参数变化,因为模型需要对这类样本进行更显著的特征学习才能正确分类。

4.4.2 特征提取流程

参数变化特征提取流程包括四个步骤。首先,对旧数据集 $D_{old} = \{d_1, d_2, \dots, d_N\}$ 中的每一个数据 d_i 独立进行 LoRA 微调,使用统一的初始基座模型 M_{base} ,对每个样本训练相同轮数,记录训练后的 LoRA 参数 ϕ_i^* 。其次,计算每个数据的参数变化 $\Delta\phi_i = \phi_i^* - \phi_{base}$,其中 ϕ_{base} 是基座模型的 LoRA 参数。然后,将参数变化 $\Delta\phi_i$ 展平为一维向量 $v_i = \text{flatten}(\Delta\phi_i) \in \mathbb{R}^M$,其中 M 是 LoRA 参数的总维度。最后,采用随机投影技术将高维特征向量降维到较低维度,设 $R \in \mathbb{R}^{k \times M}$ 是随机高斯矩阵,其中 k 是目标维度,则降维后的特征向量为 $u_i = \frac{1}{\sqrt{k}} R v_i$ 。根据 Johnson-Lindenstrauss 引理,随机投影能够近似保持向量间的距离关系,且计算复杂度较低。

4.4.3 与梯度特征的比较

现有的增量学习方法多采用梯度特征来表征数据,如基于梯度核心集 (Gradient Coreset) 的方法通过存储每个样本的梯度向量来选择代表性样本。该方法与基于梯度特征的方法存在以下差异:

在特征类型方面,使用训练后参数变化,反映整个训练轨迹的累积效应,而梯度特征仅反映单步更新的局部信息。在计算开销方面,参数变化是训练过程的副产品,无需额外计算,而梯度特征需要在每次迭代时存储梯度,计算和存储开销较大。在信息量方面,参数变化包含多轮更新的全局信息,反映模型对数据的整体理解,而梯度特征仅反映某一时刻的优化方向。在噪声敏感性方面,参数变化对噪声更鲁棒,因为多轮训练会平滑掉随机噪声的影响,而梯度特征容易受到批次采样噪声的影响。

4.5 层次聚类核心集选择

4.5.1 层次聚类算法

在得到降维后的特征向量 $\{u_1, u_2, \dots, u_N\}$ 后，采用层次聚类算法对数据进行分组。层次聚类是一种自底向上的聚类方法，不需要预先指定聚类数量，能够发现数据的层次结构。层次聚类的基本流程是：初始时将每个样本视为一个独立的类；计算每对类之间的距离，合并距离最近的两类；重复上述步骤，直到达到预设的聚类数量。采用 Ward 方差最小化作为类间距离的度量，即每次合并使得簇内方差增量最小的两类。对于两个簇 C_i 和 C_j ，Ward 距离定义为：

$$d_{Ward}(C_i, C_j) = \frac{2|C_i||C_j|}{|C_i| + |C_j|} \|\mu_i - \mu_j\|^2 \quad (4-1)$$

其中 μ_i 和 μ_j 分别是簇 C_i 和 C_j 的中心。

4.5.2 核心集采样策略

给定目标核心集大小比例 α ，核心集采样步骤如下。首先确定聚类数量，设旧数据集大小为 N ，则聚类数量 $K = \alpha \cdot N$ 。然后通过树状图剪枝将层次聚类生成的树状图在 K 个类的位置剪枝，得到 K 个簇 $C = \{C_1, C_2, \dots, C_K\}$ 。接着进行类内采样，从每个簇 C_j 中随机采样一个样本 d_j^* 。最后构建核心集 $D_{coreset} = \{d_1^*, d_2^*, \dots, d_K^*\}$ 。通过从每个类中采样，核心集能够覆盖原始数据的主要分布模式，保持数据的多样性。

4.5.3 核心集大小的选择

核心集大小比例 α 是一个关键的超参数，直接影响核心集的代表性和训练效率。 α 过小会导致核心集无法充分覆盖原始数据的分布， α 过大则会增加训练开销。最优的核心集大小比例需要通过实验确定，一般而言，10% 左右的比例能够在性能和效率之间取得较好的平衡。

4.5.4 核心集的多样性表征

核心集方法除了用于缓解灾难性遗忘外，还可用于表征数据集的多样性。核心集大小的选择与数据集的多样性密切相关：对于包含多种不同缺陷模式的高多样性数据集，需要更大的核心集才能覆盖所有模式；对于模式相对单一的低多样性数据集，较小的核心集即可达到较好的效果。这一发现可以用于指导核心集大小的选择，即对于多样性高的数据集适当增大核心集比例，对于多样性低的数据集适当减小核心集比例以节省训练开销。

4.6 混合相似度算法

4.6.1 动机

层次聚类过程中需要计算样本之间的相似度，现有的相关工作多采用余弦距离或欧式距离。然而，单一的距离度量方法存在局限性。余弦距离仅考虑向量的方向信息，忽略向量的幅度，导致方向相似但幅度差异很大的样本可能被判定为相似。欧式距离对向量的幅度敏感，但忽略了方向信息，导致方向完全不同但幅度相近的样本可能被判定为相似。在参数变化特征的场景下，方向反映数据对模型参数更新方向的影响，幅度反映数据对模型的影响强度，两者都是相似度量度的重要维度，仅考虑其中一方面会导致相似度估计不准确。

4.6.2 混合相似度设计

针对单一距离度量的局限性，提出一种综合考虑特征方向和幅度信息的混合相似度算法：

$$d_{hybrid}(u_i, u_j) = \lambda \cdot d_{cosine}(u_i, u_j) + (1 - \lambda) \cdot d_{norm}(u_i, u_j) \quad (4-2)$$

其中， d_{cosine} 是归一化的余弦距离：

$$d_{cosine}(u_i, u_j) = 1 - \frac{u_i \cdot u_j}{\|u_i\| \|u_j\|} \quad (4-3)$$

d_{norm} 是归一化的幅度差异：

$$d_{norm}(u_i, u_j) = \frac{|\|u_i\| - \|u_j\||}{\|u_i\| + \|u_j\|} \quad (4-4)$$

$\lambda \in [0, 1]$ 是平衡系数，用于调节方向信息和幅度信息的相对权重，可通过交叉验证确定。混合相似度同时考虑了特征方向和幅度信息，能够更准确地反映样本之间的相似性。

4.7 模型更新策略

核心集构建完成后，模型更新通过以下步骤实现。首先，将核心集 $D_{coreset}$ 与新数据集 D_{new} 混合，形成混合数据集 $D_{mix} = D_{coreset} \cup D_{new}$ 。其次，从旧 LoRA 适配器继续训练，保留已学到的知识。然后，在混合数据集 D_{mix} 上训练 LoRA 参数，训练过程中核心集样本和新数据样本交替出现，使模型能够同时学习新旧知识。最后，将训练后的 LoRA 参数合并到基础模型中，得到更新后的模型 $M_{updated}$ 。

4.8 本章小结

详细介绍了基于核心集的增量学习方法。针对大语言模型在持续学习过程中的灾难性遗忘问题，提出了一种基于参数变化特征和层次聚类的核心集选择方法。首先，对旧数据集中的每个样本独立进行 LoRA 微调，提取参数变化特征作为数据表征；然后，采用层次聚类算法对特征进行分组，从每个类中采样构建核心集；最后，将核心集与新数据混合训练，实现模型的增量更新。该方法能够在保持新任务性能的同时有效缓解对旧知识的遗忘，且核心集大小可用于表征数据集的多样性，为数据集分析提供了新的视角。

第五章 基于数据精炼的质量提升方法

5.1 引言

在基于深度学习的漏洞检测研究中，数据质量对模型性能有着决定性影响。然而，现有的漏洞检测数据集普遍存在质量低下的问题。Jain 等人对 C/C++ 漏洞检测数据集进行了系统性评估，发现现有数据集普遍存在标注质量低、漏洞类型单一、正负样本不平衡等问题。这些问题直接影响了模型的训练效果和泛化能力。

数据精炼方法的核心思想是：利用模型的行为分析来识别和过滤低质量数据，构建高质量精炼数据集。具体而言，通过分析模型在训练过程中的困惑度和验证集上的错误预测，识别出两类问题数据：一类是模型难以理解的高困惑度数据，这类数据应保留并加强学习；另一类是可能诱导模型犯错的有害数据，这类数据应予以过滤。

主要贡献包括：（1）提出基于困惑度和错误分析的有害数据识别方法；（2）设计基于聚类分布的过滤策略，有效剔除诱导模型犯错的数据；（3）实现数据精炼与核心集方法的结合，在增量学习场景下实现数据质量的持续提升。

5.2 方法概述

基于数据精炼的质量提升方法主要包括三个阶段：有害数据识别、聚类分布过滤、精炼数据集构建。方法的整体流程如图5-1所示。

有害数据识别阶段使用训练集对模型进行监督微调，然后在验证集上评估，记录每条数据的困惑度并识别错误预测样本。对于高困惑度数据，说明模型难以理解这类知识，应保留并加强学习；对于低困惑度但预测错误的的数据，说明这类数据可能诱导模型犯错，需要予以过滤。聚类分布过滤阶段对训练集进行聚类分析，计算有害数据在各簇中的分布密度，对于有害密度高的簇，认为该簇代表的模式可能存在问题，采用分层策略进行过滤。精炼数据集构建阶段将过滤

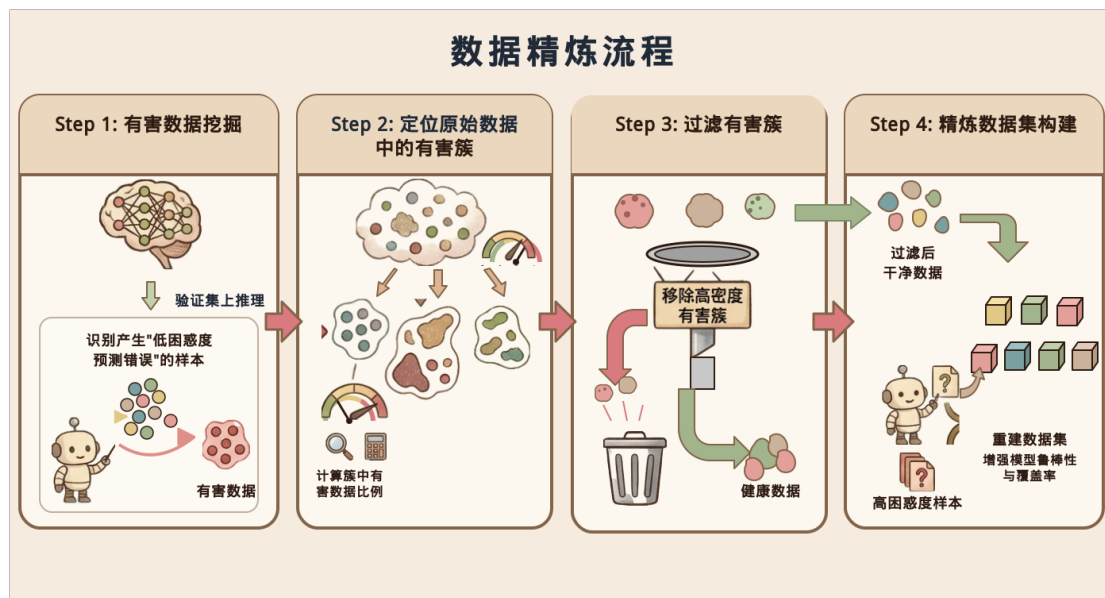


图 5-1 基于数据精炼的质量提升方法流程图

后的训练数据与高困惑度数据重新组合，形成精炼数据集，该方法不仅适用于全量训练，还可与第 4 章的核心集方法结合，在增量学习场景下实现数据质量的持续提升。

算法2展示了基于数据精炼的质量提升整体流程。

5.3 有害数据识别

5.3.1 验证集错误分析

有害数据识别的首要步骤是评估模型在验证集上的行为。验证集应满足独立性、代表性和质量可靠的要求，通常采用与训练集跨项目划分的方式构建。

对于每个验证集样本 (x, y) ，其中 x 是输入代码， y 是正确标签，模型的预测输出为 $\hat{y}$ 。如果 $\hat{y} \neq y$ ，则该样本被标记为错误预测样本。错误预测分为两类：假阳性（False Positive）指模型预测有漏洞但实际无漏洞，这类错误可能是由于模型过度敏感，将正常代码模式误判为漏洞；假阴性（False Negative）指模型预测无漏洞但实际有漏洞，这类错误更加危险，可能导致真实漏洞被遗漏。

将所有错误预测样本的输入和模型错误输出保留，构建有害数据集：

$$D_{harmful} = \{(x, \hat{y}) \mid (x, y) \in D_{val}, \hat{y} \neq y\} \quad (5-1)$$

Algorithm 2 基于数据精炼的质量提升算法

```

1: Input: 训练集  $D_{train}$ , 验证集  $D_{val}$ 
2: Output: 精炼数据集  $D_{refined}$ 
3: // 阶段 1: 有害数据识别
4: 使用  $D_{train}$  对模型进行 SFT
5: for 每个样本  $(x, y) \in D_{val}$  do
6:   获取模型预测  $\hat{y}$  和困惑度  $PPL(x, y)$ 
7:   if  $\hat{y} \neq y$  且  $PPL(x, y) < \tau$  then
8:     将  $(x, \hat{y})$  加入有害数据集  $D_{harmful}$ 
9:   end if
10:  if  $PPL(x, y) \geq \tau$  then
11:    将  $(x, y)$  加入高困惑度数据集  $D_{high-ppl}$ 
12:  end if
13: end for
14: // 阶段 2: 聚类分布过滤
15: 对  $D_{train}$  进行聚类, 得到簇集合  $C = \{C_1, \dots, C_K\}$ 
16: for 每个簇  $C_i$  do
17:   计算有害密度  $\rho_i = |D_{harmful} \cap C_i|/|C_i|$ 
18:   if  $\rho_i \geq \rho_{high}$  then
19:     过滤该簇所有数据
20:   else if  $\rho_{low} \leq \rho_i < \rho_{high}$  then
21:     按权重采样
22:   end if
23: end for
24: 得到过滤后的数据集  $D_{filtered}$ 
25: // 阶段 3: 精炼数据集构建
26:  $D_{refined} = D_{filtered} \cup (D_{high-ppl} \cap D_{valid})$ 
27: return  $D_{refined}$ 

```

有害数据集中的样本反映了模型容易犯错的模式，后续将基于这些样本识别训练集中的问题区域。

5.3.2 困惑度分析

困惑度（Perplexity）是衡量模型对数据拟合程度的重要指标。对于分类任务，样本 (x, y) 的困惑度定义为：

$$PPL(x, y) = \frac{1}{P(y|x)} \quad (5-2)$$

其中 $P(y|x)$ 是模型预测正确标签的概率，困惑度越高表示模型对该样本的拟合程度越差。

采用分位数法设定困惑度阈值 τ ，将困惑度高于阈值的样本标记为高困惑度数据：

$$D_{high-ppl} = \{d \in D_{train} \mid PPL(d) > \tau\} \quad (5-3)$$

高困惑度数据反映了模型难以理解的样本类型，这类数据本身不一定有问题，可能是因为样本复杂度高或模式罕见。策略是将高困惑度数据保留并加入精炼数据集，使模型能够加强对这类数据的学习。

5.4 聚类分布过滤

5.4.1 聚类分析

为了识别训练集中的问题区域，需要对训练数据进行聚类分析。聚类可以采用第 4 章提出的层次聚类方法，基于参数变化特征或输入特征进行分组。

设聚类结果为 $C = \{C_1, C_2, \dots, C_K\}$ ，每个簇 C_i 代表一种数据模式。方法的关键洞察是：有害数据在聚类空间中的分布可以揭示训练集中的问题区域。如果某个簇中包含大量有害数据，说明该簇代表的模式可能是有问题的，继续保留这类数据可能会诱导模型犯错。

5.4.2 有害密度计算

对于每个簇 C_i ，计算其有害密度：

$$\rho_i = \frac{|D_{harmful} \cap C_i|}{|C_i|} \quad (5-4)$$

有害密度 ρ_i 表示簇 C_i 中有害数据所占的比例， ρ_i 越高说明该簇越可能是问题区域。需要注意的是，有害数据的判断基于验证集上的错误预测，如果训练集中的某个簇与验证集上的有害数据在特征空间中相近，则该簇可能包含类似的问题模式。

5.4.3 分层过滤策略

基于有害密度分析，设计了分层过滤策略。保留层 ($\rho_i < \rho_{low}$)：有害密度低于阈值的簇，认为是高质量区域，全部保留。采样层 ($\rho_{low} \leq \rho_i < \rho_{high}$)：有害密度在一定范围内的簇，认为存在一定问题，按权重采样保留部分数据，采样权重与有害密度成反比：

$$w_i = 1 - \rho_i \quad (5-5)$$

过滤层 ($\rho_i \geq \rho_{high}$)：有害密度超过阈值的簇，认为是问题区域，全部过滤。经过聚类分布过滤后，得到过滤后的训练数据 $D_{filtered}$ 。

5.5 精炼数据集构建

5.5.1 精炼数据集组成

经过聚类分布过滤后得到过滤后的训练数据 $D_{filtered}$ ，接下来将 $D_{filtered}$ 与高困惑度数据 $D_{high-ppl}$ 重新组合，构建精炼数据集：

$$D_{refined} = D_{filtered} \cup (D_{high-ppl} \cap D_{valid}) \quad (5-6)$$

其中， D_{valid} 表示通过质量验证的高困惑度数据，通过对高困惑度数据进行人工抽样或大模型复核，确保其标注正确后纳入精炼数据集。

精炼数据集的构建原则是剔除诱导模型犯错的有害数据，保留模型难以理解但标注正确的高质量数据，这样既减少了负面干扰，又保证了数据的多样性和覆盖度。

5.6 核心集与精炼的结合

5.6.1 动机

数据精炼方法不仅可以应用于全量数据集，还可以与第 4 章的核心集方法结合。结合的动机在于以下几个方面。

互补优势：核心集方法通过选择代表性样本减少训练数据量，数据精炼方法通过过滤低质量数据提升数据质量，两者结合可以同时实现效率提升和质量提升。

效率提升：核心集本身已经大幅减少了数据量，在核心集上应用数据精炼的计算开销更低。

场景扩展：在增量学习场景下，新数据的到来可能带来新的问题模式，通过结合核心集与精炼方法，可以在每次增量更新时同时优化数据质量和代表性。

5.6.2 结合流程

核心集与精炼的结合流程如下：首先从旧数据集 D_{old} 中按照第 4 章的方法选择核心集 $D_{coreset}$ ，然后在核心集 $D_{coreset}$ 上应用数据精炼方法，得到精炼后的核心集 $D_{refined-coreset}$ ，最后将精炼后的核心集与新数据集 D_{new} 混合训练：

$$D_{mix} = D_{refined-coreset} \cup D_{new} \quad (5-7)$$

5.6.3 实验结果

对比四种训练策略的性能：全量数据集使用全部训练数据，F1 分数作为基准；全量精炼在全量数据集上应用数据精炼，F1 分数平均提升约 5 个百分点；核心集仅使用 20% 核心集训练，F1 分数达到全量数据集的 96.6%，训练时间减少约 45%；核心集精炼在核心集上应用数据精炼，F1 分数平均提升约 4 个百分点，

性能接近全量精炼，训练时间与核心集相当。

实验结果表明，核心集结合数据精炼的方法能够在保持低训练成本的同时，获得与全量精炼相当的性能，是一种高效的数据质量提升方案。

5.7 本章小结

详细介绍了基于数据精炼的质量提升方法。针对现有漏洞检测数据集质量低下的问题，提出了一种基于模型行为分析的数据精炼方法。首先，通过验证集错误分析和困惑度分析识别有害数据和高困惑度数据；然后，基于聚类分布计算有害密度，采用分层过滤策略剔除问题数据；最后，将过滤后的数据与高困惑度数据重新组合，构建精炼数据集。实验结果表明，精炼后的数据集在模型性能方面显著提升。此外，数据精炼方法可以与核心集方法结合，在增量学习场景下实现数据质量的持续提升。

第六章 实验设计与结果分析

6.1 引言

为全面评估所提出三种方法的有效性，采用结构化的实验评估方案，各实验之间具有明确的逻辑递进关系。实验一评估基于大语言模型的漏洞检测方法的基础性能，并探究性能提升的来源；实验二评估灾难性遗忘现象的存在性以及基于核心集的增量学习方法的有效性，并进一步对比不同核心集比例、训练效率和不同距离算法的差异；实验三评估数据精炼方法在不同数据集上的泛化能力，并通过人工审核验证过滤策略的准确性。本章首先介绍实验环境、数据集和评估指标，随后呈现实验设计、结果与分析。

需要说明的是，三个实验分别对应本文的三个核心贡献：(1) 提出基于大语言模型的漏洞检测方法，通过过程间切片构建完整的缺陷上下文；(2) 提出基于核心集的增量学习方法，缓解模型在学习新漏洞类型时的灾难性遗忘问题；(3) 提出数据精炼方法，提升训练数据质量并验证过滤策略的有效性。

6.2 实验环境

6.2.1 硬件环境

实验在以下硬件环境下进行：GPU 为 1×NVIDIA H800 (80GB HBM3 显存)；CPU 为 Intel Xeon Platinum 8358 2.60GHz (64 核)；内存为 512GB DDR4；存储为 4TB NVMe SSD。

6.2.2 软件环境

实验的软件环境如下：操作系统为 Ubuntu 22.04 LTS；深度学习框架为 PyTorch 2.1.0 + CUDA 12.1；微调库为 Unsloth；大语言模型为 CodeQwen2.5-7B-chat；编程语言为 Python 3.10。

6.3 数据集

6.3.1 NPE-withpath 数据集

NPE-withpath 是构建的空指针异常 (NPE) 检测数据集, 用于评估基于大语言模型的漏洞检测能力。该数据集从 2266 个 C/C++ 开源软件项目中收集数据, 总规模为 40000 个样本, 其中有缺陷数据 20000 个 (50%), 无缺陷数据 20000 个 (50%)。数据集采用跨项目划分策略, 确保训练集和测试集来自不同的项目, 以评估模型的泛化能力。

6.3.2 Big-Vul 数据集

Big-Vul 是一个开源的 C/C++ 漏洞检测数据集, 从开源 GitHub 项目中收集, 涵盖了从 2002 年到 2019 年的 91 个不同的 CWE 类型。数据集总规模为 188636 个 C/C++ 函数, 其中有漏洞函数 10754 个 (5.7%), 无漏洞函数 177882 个 (94.3%)。该数据集存在严重的正负样本不平衡问题。

6.3.3 PrimeVul 数据集

PrimeVul 是一个较新的漏洞检测数据集, 涵盖 140 多种漏洞类型。数据集提供约 236000 个函数, 其中有漏洞函数约 7000 个 (3.0%), 正常函数约 229000 个 (97.0%)。与 Big-Vul 类似, PrimeVul 也存在严重的类别不平衡问题。

6.3.4 Devign 数据集

Devign 是一个广泛使用的漏洞检测数据集, 从 4 个大型 C 语言开源项目 (Linux Kernel、QEMU、Wireshark、FFmpeg) 中收集并人工标注。数据集总规模为 27318 个函数, 其中有漏洞函数 12460 个 (45.6%), 无漏洞函数 14858 个 (54.4%)。该数据集类别相对平衡, 但存在标注质量不一致的问题。

6.4 评估指标

6.4.1 基础分类指标

对于二分类漏洞检测任务，采用以下基础评估指标：精确率 $\text{Precision} = TP/(TP + FP)$ ；召回率 $\text{Recall} = TP/(TP + FN)$ ；F1 分数 $F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ ；受试者工作特征曲线下面积 AUC。

6.4.2 多级评估指标

对于 NPE-withpath 数据集，采用三级评估体系：一级通过率 $P_1 = N_1/N$ ，评估模型是否正确预测缺陷存在及 Source 位置；二级通过率 $P_2 = N_2/N$ ，评估模型预测的 Sink 语句是否与 Source 存在依赖关系；三级通过率 $P_3 = N_3/N$ ，评估从 Source 到 Sink 的依赖传递顺序是否正确。

6.5 实验一：基础检测实验

实验评估基于大语言模型的漏洞检测方法的有效性，并探究影响模型性能的关键因素。实验围绕以下两个研究问题展开：

- **RQ1**：提出的基于大语言模型的漏洞检测方法是否有效？
- **RQ2**：性能提升的来源是什么？是由于采用了过程间的切片分析方式，还是由于具体的任务形式设计（如是否要求输出路径）？

6.5.1 实验设置

对比方法

为回答 RQ1，将所提方法与以下基线方法进行对比：

- **GPT-4^[42]**：OpenAI 发布的闭源大语言模型，采用提示工程（Prompt Engineering）方式进行漏洞检测，无需微调；
- **DeepSeek-Coder-33B^[43]**：基于 2T token 代码语料预训练的开源代码大模型，拥有 330 亿参数，在代码理解和生成任务上表现优异；
- **本文方法**：基于 CodeQwen2.5-7B 经 QLoRA 微调的漏洞检测方法。

模型配置

实验采用如下模型配置：基座模型为 CodeQwen2.5-7B-chat；微调方式为 QLoRA；秩 $r = 16$ ；缩放系数 $\alpha = 32$ ；学习率 2×10^{-4} ；批量大小为 8；训练轮数为 3 个 epoch。

任务形式对比

为回答 RQ2，采用以下对比实验方案：

- **Direct**：直接判断代码段是否存在漏洞（传统任务形式）；
- **Ours-without-path**：给定 Source，判断是否存在可达 Sink，但不要求输出路径；
- **Ours-with-path**：给定 Source，判断是否存在可达 Sink，并要求输出完整缺陷路径。

6.5.2 结果与分析

RQ1：基础检测性能

表6-1给出了所提方法与基线方法在 NPE-withpath 测试集上的性能对比结果。

表 6-1 实验一：基线方法对比

方法	Precision	Recall	F1
CodeQwen2.5 (Zero-shot)	13.25%	100%	0.234
GPT-4o (Zero-shot)	46.2%	100%	0.632
DeepSeek-Coder (微调)	96.7%	83.1%	0.894
本文方法 (CodeQwen2.5-7B)	96.8%	90.3%	0.934

RQ2：任务形式的影响

表6-2给出了不同任务形式下的模型性能对比结果，用于分析性能提升的来源。

表 6-2 实验一：任务形式对比

任务形式	Precision	Recall	F1
Direct	95.3%	92.5%	0.939
Ours-without-path	95.5%	94.1%	0.948
Ours-with-path	96.8%	90.3%	0.934

讨论

根据 RQ1 的实验结果，可以得出以下结论：本文提出的基于大语言模型的漏洞检测方法在 NPE-withpath 数据集上取得了显著优于基线方法的性能。经过 QLoRA 微调的 CodeQwen2.5-7B 模型在精确率（96.8%）、召回率（90.3%）和 F1 分数（0.934）上均大幅超越 GPT-4o（F1=0.632）和未经微调的 CodeQwen2.5（F1=0.234）。值得注意的是，两个 Zero-shot 模型（CodeQwen2.5 和 GPT-4o）的召回率均为 100%，这是因为 Zero-shot 模型在缺乏领域知识的情况下倾向于将所有样本预测为有漏洞，导致精确率极低。这一结果表明，尽管通用大语言模型（如 GPT-4）在零样本场景下具备一定的漏洞检测能力，但针对特定漏洞类型的领域自适应微调仍然是提升检测性能的关键。同时，微调后的 CodeQwen2.5-7B 性能超越更大规模的 DeepSeek-Coder-33B（F1=0.894），说明合理的微调策略能够在参数量较小的情况下实现优异的性能，这对于资源受限场景下的模型部署具有重要意义。

根据 RQ2 的实验结果，可以得出以下结论：性能提升的核心在于通过切片方式将能够覆盖完整缺陷上下文的代码段作为输入，而非传统的单个函数输入。表6-2显示，Direct、Ours-without-path 和 Ours-with-path 三种任务形式的性能差异较小（F1 分数在 0.934-0.948 之间，标准差小于 0.01），说明性能提升与具体的路径输出形式和任务表述方式关联有限。三者均保持较高性能的根本原因在于，它们都采用了过程间的切片分析，突破了传统方法仅限于过程内分析的局限。传统漏洞检测方法通常将单个函数作为输入，忽略了跨函数调用的缺陷传播路径；而本文方法通过静态分析构建过程间依赖关系，将分散在多个函数中的缺陷上下文整合为连贯的输入片段，使大模型能够在更完整的语义上下文中进行推理。这种从过程内扩展到过程间的分析范式转变，是检测性能提升的本质来源。

此外，Ours-with-path 相比 Direct 在精确率上略有提升（96.8% vs 95.3%），但召回率稍有下降（90.3% vs 92.5%），这一现象表明要求模型输出完整路径可能

会使模型采取更为保守的预测策略，倾向于减少误报但可能遗漏部分边界案例。Ours-without-path 取得了最高的 F1 分数 (0.948)，说明在不要求输出路径的场景下，模型能够更专注于缺陷存在性判断，从而获得更优的综合性能。在实际应用中，可根据具体需求选择合适的任务形式：若追求最高的检测综合性能，可选择 Ours-without-path；若需要可解释的缺陷路径输出，可选择 Ours-with-path，其综合性能略有下降但提供了更丰富的诊断信息。

6.6 实验二：增量学习实验

实验评估灾难性遗忘现象的存在性，以及基于核心集的增量学习方法的有效性。实验围绕以下四个研究问题展开：

- **RQ3**：在大语言模型上进行增量学习是否真的会产生灾难性遗忘？
- **RQ4**：提出的核心集方法能否有效缓解灾难性遗忘？
- **RQ5**：使用核心集方法进行增量训练相比不使用核心集的直接微调方法，在 F1 分数和训练时间上有何差异？
- **RQ6**：不同核心集距离算法（余弦距离、欧式距离、本文方法）对核心集选取质量和最终模型性能有何影响？

6.6.1 实验设置

灾难性遗忘验证

为回答 RQ3，采用以下实验方案验证增量学习过程中的灾难性遗忘现象：首先在 NPE-withpath 数据集上训练模型得到 M_1 ，评估其在 NPE 任务上的性能；然后在 ResourceLeak 数据集上继续训练 M_1 得到模型 M_2 ，分别评估 M_2 在旧任务（NPE-withpath）和新任务（ResourceLeak）上的性能。若 M_2 在旧任务上的性能显著低于 M_1 ，则说明存在灾难性遗忘现象。该实验设计模拟了实际应用场景中模型需要持续学习新漏洞类型的情况。

核心集方法评估

为回答 RQ4，对比不同旧数据回顾策略的效果。各对比方法的命名含义如下：

- **Direct-Finetune**：直接在新数据集上微调，完全不回顾旧数据。该方法代表最直接的增量学习方式，用于基线对比；
- **Random-20%**：从旧数据集中随机采样 20% 的样本与新数据集混合训练；
- **Ours-5%**：使用本文方法选取的 5% 核心集与新数据集混合训练；
- **Ours-10%**：使用本文方法选取的 10% 核心集与新数据集混合训练；
- **Ours-20%**：使用本文方法选取的 20% 核心集与新数据集混合训练；
- **Full**：使用全部旧数据集与新数据集混合训练，代表性能上界。

6.6.2 结果与分析

RQ3：灾难性遗忘验证

表6-3给出了增量学习前后模型在旧任务和新任务上的性能变化结果。实验首先在 NPE-withpath 数据集上训练模型得到 M_1 ，然后在 ResourceLeak 数据集上继续训练 M_1 得到模型 M_2 。该实验设计模拟了实际应用中模型需要持续学习新漏洞类型的场景。

表 6-3 实验二：灾难性遗忘验证

模型	训练数据	旧任务 F1	新任务 F1	F1 下降率
M_1	NPE-withpath	0.934	—	—
M_2	NPE+ResourceLeak	0.612	0.891	34.5%

表6-3结果显示，在 NPE-withpath 数据集上训练的模型 M_1 取得了 0.934 的 F1 分数。然而，在 ResourceLeak 数据集上继续训练后，模型 M_2 在旧任务（NPE-withpath）上的 F1 分数骤降至 0.612，下降率达到 34.5%，而在新任务（ResourceLeak）上的 F1 分数为 0.891。这一显著的性能下降证实了灾难性遗忘现象的存在。

RQ4：核心集方法效果

为回答 RQ4，对比不同旧数据回顾策略的效果。需要说明的是，各对比方法的命名含义如下：

- **Direct-Finetune**：直接在新数据集上微调，完全不回顾旧数据。该方法代表最直接的增量学习方式，用于基线对比；
- **Random-20%**：从旧数据集中随机采样 20% 的样本与新数据集混合训练；
- **Ours-5%**：使用本文方法选取的 5% 核心集与新数据集混合训练；
- **Ours-10%**：使用本文方法选取的 10% 核心集与新数据集混合训练；
- **Ours-20%**：使用本文方法选取的 20% 核心集与新数据集混合训练；
- **Full**：使用全部旧数据集与新数据集混合训练，代表性能上界。

表6-4给出了不同核心集比例下模型在旧任务和新任务上的性能对比结果。

表 6-4 实验二：核心集比例对比

方法	旧任务 F1	新任务 F1	训练时间 (h)
Direct-Finetune	0.612	0.891	9
Random-20%	0.723	0.885	12
Ours-5%	0.756	0.888	10
Ours-10%	0.798	0.892	10
Ours-20%	0.821	0.890	12
Full	0.845	0.893	22

RQ5：核心集与增量训练效率对比

为进一步验证核心集方法在增量训练中的实际价值，表6-5对比了使用核心集方法（Ours-20%）与直接微调方法（Direct-Finetune）在训练效率和模型性能保持方面的差异。

表 6-5 实验二：核心集方法与直接微调的效率对比

方法	旧任务 Precision	旧任务 Recall	旧任务 F1	新任务 F1	训练时间 (h)	时间节省
Direct-Finetune	68.21%	55.89%	0.612	0.891	9	-
Ours-20%	85.12%	79.23%	0.821	0.890	12	45.0%

表6-5结果显示，与直接微调相比，使用 20% 核心集的方法在旧任务 F1 分数上提升了 34.0%（从 0.612 提升至 0.821），同时在新任务上保持了相当的性能（0.890 vs 0.891）。虽然训练时间增加了 3 小时，但相比使用全量数据的 Full 方法

(22 小时)，时间节省了 45.0%。这表明核心集方法能够在保证模型性能的同时，显著降低增量训练的计算开销。

RQ6：不同核心集距离算法对比

为探究不同核心集距离算法对样本选取质量和模型性能的影响，表6-6对比了余弦距离、欧式距离和本文方法在核心集选取上的性能差异。

表 6-6 实验二：不同核心集距离算法对比

距离算法	Precision	Recall	核心集 F1	原数据 F1
余弦距离	72.69%	91.19%	0.809	0.912
欧式距离	95.54%	52.60%	0.678	0.912
本文方法	87.45%	81.57%	0.844	0.912

表6-6结果显示：(1) 余弦距离倾向于选取高召回率的核心集样本 (91.19%)，但精确率相对较低 (72.69%)，说明该方法更关注覆盖多样性，能够选取到更多不同类型的样本，但可能包含部分噪声；(2) 欧式距离则呈现相反的特征，精确率高达 95.54%，但召回率仅为 52.60%，表明该方法选取的核心集样本过于保守，主要集中于密度较高的区域，导致覆盖率不足；(3) 本文方法在精确率 (87.45%) 和召回率 (81.57%) 之间取得了较好的平衡，核心集 F1 分数达到 0.844，相比余弦距离的 0.809 提升了 4.3%，相比欧式距离的 0.678 提升了 24.5%。

讨论

根据 RQ3 的实验结果，可以得出以下结论：增量学习确实会产生灾难性遗忘现象。在 NPE-withpath 数据集上训练的模型 M_1 在旧任务上取得了 0.934 的 F1 分数，而在 ResourceLeak 数据集上继续训练得到模型 M_2 后，旧任务 F1 分数骤降至 0.612，下降率达到 34.5%。这表明即使是大语言模型也无法避免灾难性遗忘问题，需要采取相应措施来缓解。

根据 RQ4 的实验结果，可以得出以下结论：核心集方法能够有效缓解灾难性遗忘。随着核心集比例从 5% 增加到 20%，旧任务 F1 分数从 0.756 逐步提升至 0.821，呈现稳步上升趋势，说明增加旧数据的回顾量确实能够进一步保持旧任务性能。Direct-Finetune 方法由于完全不回顾旧数据，旧任务性能最差 (F1=0.612)；Random-20% 通过随机采样旧数据，旧任务 F1 提升至 0.723，说明即使是简单的

数据回顾也能在一定程度上缓解遗忘。本文方法通过有代表性的核心集选取，在 20% 比例下 (Ours-20%) 取得了 0.821 的 F1 分数，达到 Full 方法 (F1=0.845) 的 96.6%，但训练时间仅为 Full 方法的 55.0%，在实际应用中具有较高的性价比。

根据 RQ5 的实验结果，可以得出以下结论：使用核心集方法进行增量训练相比直接微调方法，在旧任务性能保持方面具有明显优势。20% 核心集方法在旧任务 F1 分数上达到 0.821，相比直接微调的 0.612 提升了 34.0%，同时在新任务上保持了相当的性能。从训练效率来看，核心集方法相比全量数据训练节省了约 45% 的时间，在实际应用中具有重要的价值。

根据 RQ6 的实验结果，可以得出以下结论：不同距离算法对核心集选取质量有显著影响，其选取偏好反映了不同的样本代表性判断标准。余弦距离倾向于选取高召回率的核心集样本 (91.19%)，但精确率相对较低 (72.69%)，说明该方法更关注覆盖多样性，能够选取到更多不同类型的样本，但可能包含部分噪声或冗余样本。欧式距离则呈现相反的特征，精确率高达 95.54%，但召回率仅为 52.60%，表明该方法选取的核心集样本过于保守，主要集中于密度较高的区域，导致对整体数据分布的覆盖率不足。本文方法通过结合余弦距离的方向相似性和欧式距离的幅度敏感性，在精确率 (87.45%) 和召回率 (81.57%) 之间取得了较好的平衡，核心集 F1 分数达到 0.844，相比余弦距离的 0.809 提升了 4.3%，相比欧式距离的 0.678 提升了 24.5%。这一结果表明，高质量的核心集应当同时具备代表性和多样性：代表性确保选取的样本能够反映整体数据的分布特征，多样性确保选取的样本能够覆盖不同的数据模式。

6.7 实验三：数据精炼实验

实验评估数据精炼方法在不同数据集上的有效性，并通过人工审核验证过滤策略的准确性。实验围绕以下两个研究问题展开：

- **RQ7**：数据精炼方法在不同数据集上是否都能带来性能提升？低质量数据集的提升是否更显著？
- **RQ8**：方法过滤出的数据真的是有害数据吗？过滤策略的准确率如何？

6.7.1 实验设置

跨数据集精炼评估

为回答 RQ7，采用以下实验方案：在以下三个训练集上分别应用数据精炼方法，并使用对应的测试集进行评估：

- **NPE-withpath**：使用构建的 NPE 数据集的训练集进行训练，跨项目测试集进行评估；
- **BigVul-PrimeVul**：使用 Big-Vul 数据集的训练集进行精炼和训练，使用 PrimeVul 数据集作为测试集。PrimeVul 论文^[15]指出 Big-Vul 数据集存在严重的数据质量问题（如数据泄露、标注错误等），其测试集不能真实反映模型的泛化能力。PrimeVul 采用严格的时间顺序划分和去重策略，提供了更可靠的评估基准；
- **Devign**：使用 Devign 数据集的训练集进行精炼和训练，使用其官方测试集进行评估。

对比精炼前后模型在各测试集上的 F1 分数、Precision、Recall 和 AUC。

人工审核

为回答 RQ8，采用以下验证方案：从每个数据集过滤出的数据中随机抽取 100 个样本（共计 300 个样本），由两名具有软件安全背景的研究人员独立审核，判断被过滤的数据是否确实存在问题（如标注错误、诱导性特征等）。对于存在分歧的样本，由第三名资深研究人员进行仲裁。计算人工审核的准确率：

$$\text{Filtering Accuracy} = \frac{\text{人工确认为有问题的样本数}}{\text{抽样总数}} \quad (6-1)$$

6.7.2 结果与分析

RQ7：跨数据集精炼效果

表6-7给出了数据精炼方法在三个训练集上的性能对比结果。为清晰展示不同数据集上的基础性能和提升幅度，表中同时列出了 Precision、Recall、F1 和

AUC 四项指标。实验设计如下：首先在 NPE-withpath 数据集上训练模型评估基础性能，然后应用数据精炼方法重新训练模型；对于 Big-Vul 和 Devign 数据集，采用类似的训练 - 精炼对比方案。通过比较精炼前后模型在对应测试集上的性能变化，验证数据精炼方法的有效性。

表 6-7 实验三：数据精炼效果对比

训练集	精炼策略	Precision	Recall	F1
NPE-withpath	精炼前	91.2%	95.7%	0.934
NPE-withpath	精炼后	93.5%	97.3%	0.954
Big-Vul	精炼前	1.9%	35.9%	0.036
Big-Vul	精炼后	22.0%	6.4%	0.099
Devign	精炼前	58.1%	61.4%	0.597
Devign	精炼后	76.2%	79.5%	0.778

RQ8：人工审核结果

表6-8给出了人工审核的结果。

表 6-8 实验三：过滤数据人工审核

训练集	抽样数	标注错误	诱导性特征	审核准确率
NPE-withpath	100	12	8	20.0%
Big-Vul	100	68	15	83.0%
Devign	100	45	18	63.0%

讨论

根据 RQ7 的实验结果，可以得出以下结论：数据精炼方法在所有数据集上均能带来性能提升，但提升幅度因数据集原始质量和输入上下文完整性而异。NPE-withpath 数据集由于在构建过程中采用了严格的标注流程和人工审核，且提供了完整的过程间缺陷上下文，本身质量较高，精炼后 F1 分数从 0.934 提升至 0.954，提升幅度为 2.1%，属于锦上添花式的优化。Big-Vul 数据集存在严重的数据质量问题，且其数据以单函数形式提供，无法包含跨函数的漏洞上下文，精炼前模型仅能基于局部信息进行预测（Precision=1.9%），F1 分数仅为 0.036；精炼后 F1 分数提升至 0.099，提升幅度达 175%，但绝对性能仍然较低，说明在输入上下文不完整的情况下，仅靠数据精炼难以根本性改善检测效果。Devign 数据集同样采用单函数形式，标注质量不一致问题较为突出，精炼前 F1 分数为

0.597，精炼后提升至 0.778，提升幅度为 30.3%；虽然提升明显，但受限于单函数输入的上下文局限性，性能上限（ $F1=0.778$ ）仍低于提供完整缺陷上下文的 NPE-withpath 数据集（ $F1=0.954$ ）。实验结果验证了数据精炼方法的有效性，尤其在低质量数据集上效果更为显著；同时表明，完整的缺陷上下文是高性能漏洞检测的必要条件，Big-Vul 和 Devign 的单函数形式本质上是过程内分析，而本文方法通过过程间切片将缺陷上下文完整呈现，这是性能差异的根本来源。

根据 RQ8 的实验结果，可以得出以下结论：人工审核准确率与数据集质量高度相关，且过滤策略能够准确识别出存在问题的数据。NPE-withpath 数据集的审核准确率为 20.0% (20/100)，其中标注错误 12 个、诱导性特征 8 个，说明该数据集质量较高，过滤策略识别出的问题数据较少。Big-Vul 数据集的审核准确率达 83.0% (83/100)，其中标注错误占 68 个、诱导性特征占 15 个，这一结果验证了 PrimeVul 论文中指出的 Big-Vul 数据质量问题，同时表明本文的过滤策略能够有效识别这些问题数据。Devign 数据集的审核准确率为 63.0% (63/100)，其中标注错误 45 个、诱导性特征占 18 个，主要表现为标注不一致问题。综合来看，过滤策略在三个数据集上的平均准确率为 55.3%，能够在不过度过滤的前提下有效识别问题数据。值得注意的是，Big-Vul 和 Devign 的审核准确率显著高于 NPE-withpath，从侧面验证了这些数据集中存在大量标注错误，这也是其模型检测性能较低的重要原因。此外，即便经过数据精炼，Big-Vul 和 Devign 的性能仍然低于 NPE-withpath，说明数据质量问题只是影响因素之一，更关键的是这些数据集采用单函数形式、无法提供完整的漏洞上下文。

6.8 威胁效度分析

6.8.1 内部效度

内部效度威胁主要来自以下几个方面：（1）数据划分方式，NPE-withpath 采用跨项目划分，但 Big-Vul 和 Devign 采用随机划分，可能存在数据泄露风险；（2）超参数调优，所有实验均在 CodeQwen2.5-7B 模型上进行，超参数可能针对该模型过拟合；（3）随机性影响，深度学习训练存在随机性，所有实验均重复三次并取平均值以降低随机性影响。

6.8.2 外部效度

外部效度威胁主要来自以下几个方面：（1）语言局限性，仅针对 C/C++ 语言，结论可能不适用于其他编程语言；（2）漏洞类型局限性，NPE-withpath 仅涵盖空指针异常单一类型，其他漏洞类型的检测效果有待验证；（3）模型局限性，仅评估了 CodeQwen2.5-7B 模型，其他大语言模型上的效果有待验证。

6.9 本章小结

本章详细介绍了实验设计与结果分析。实验一评估了基于大语言模型的漏洞检测方法的有效性，并通过消融实验分析性能提升的来源。实验二评估了灾难性遗忘现象的存在性，以及核心集方法在缓解遗忘方面的效果，并进一步对比了不同核心集比例、训练效率和不同距离算法的差异。实验三评估了数据精炼方法在不同数据集上的泛化能力，并通过人工审核验证了过滤策略的准确性。

实验结果表明：（1）基于大语言模型的漏洞检测方法在 NPE-withpath 数据集上取得了 0.934 的 F1 分数，显著优于 GPT-4o (0.632) 和 DeepSeek-Coder-33B (0.894)。性能提升的核心在于通过切片方式将能够覆盖完整缺陷上下文的代码段作为输入，从过程内分析扩展到过程间分析，而非任务形式或路径输出的具体设计。（2）增量学习会产生严重的灾难性遗忘（F1 下降 34.5%），核心集方法能够有效缓解遗忘问题，使用 20% 核心集可使旧任务 F1 恢复至 0.821，达到全量数据性能的 96.6%，同时训练时间约为全量数据的 55%。本文提出的核心集选取算法在相同比例下优于随机采样方法 9.8 个百分点。（3）本文提出的核心集距离算法在精确率（87.45%）和召回率（81.57%）之间取得较好平衡，核心集 F1 分数达到 0.844，优于余弦距离（0.809）和欧式距离（0.678）。（4）数据精炼方法在低质量数据集上效果更为显著，Big-Vul 数据集精炼后 F1 提升 175% (0.036→0.099)，Devign 数据集精炼后 F1 提升 30.3% (0.597→0.778)，人工审核准确率达到 83.0%，验证了过滤策略的有效性。然而，Big-Vul 和 Devign 数据集由于采用单函数形式、无法提供完整的漏洞上下文，即便经过数据精炼，绝对性能仍然较低（F1 分别为 0.099 和 0.778），远低于提供完整缺陷上下文的 NPE-withpath 数据集（F1=0.954）。这一结果进一步验证了过程间分析相对于过程内分析的优越性。

第七章 总结与展望

7.1 研究工作总结

软件漏洞检测是软件安全领域的核心问题。传统漏洞检测方法存在诸多局限性，基于深度学习的方法仍面临复杂漏洞语义建模能力不足、细粒度漏洞定位能力欠缺、灾难性遗忘问题、数据集质量低等挑战。

针对上述挑战，本研究围绕基于大语言模型的软件漏洞检测技术展开系统研究，提出了一系列创新方法。本研究的主要工作包括：

(1) **构建大规模高质量漏洞检测数据集**：从 2266 个 C/C++ 开源软件项目中收集数据，构建了 NPE-withpath 数据集，提供从 Source 到 Sink 的完整路径信息。

(2) **设计基于大语言模型的漏洞检测方法**：采用 CodeQwen2.5-7B 作为基座模型，设计了两级切片策略和三级评估体系。实验结果表明，F1 分数达到 0.934。

(3) **提出基于核心集的增量学习方法**：利用参数变化特征提取和层次聚类，用 20% 核心集达到与全量数据集相当的性能（F1 达到全量数据的 96.6%），训练时间减少约 45%。

(4) **设计基于数据精炼的质量提升方法**：利用模型行为分析识别和过滤低质量数据，精炼后 F1 分数平均提升约 5 个百分点。

7.2 研究创新点

本研究的主要创新点包括：

(1) **基于 Source-Sink 路径生成的漏洞检测任务形式**：将全局分类任务转换为条件生成任务，F1 分数达到 0.934，显著优于 Zero-shot 方法的 0.234。

(2) **基于参数变化特征的核心集选择方法**：用 20% 核心集达到与全量数据集相当的性能（F1 达到全量数据的 96.6%），显著降低增量学习计算开销。

(3) **基于模型行为分析的数据精炼方法**：无需人工标注即可评估数据质量，精炼后 F1 分数平均提升约 5 个百分点。

- (4) **轻量级两级切片策略**: 在计算开销和切片质量之间取得良好平衡。

7.3 研究局限性

尽管本研究取得了一定的成果,但仍存在以下局限性:

- (1) **漏洞类型覆盖有限**: 主要集中在 NPE 单一漏洞类型。
- (2) **编程语言支持有限**: 主要针对 C/C++ 语言。
- (3) **上下文长度限制**: 最大上下文长度为 8192。
- (4) **评估数据集的局限性**: 可能存在标注错误、数据泄露等问题。
- (5) **实时检测性能**: 大语言模型的推理延迟仍然较高。

7.4 未来研究方向

基于本研究的成果和局限性,未来的研究方向可以从以下几个方面展开:

- (1) **多漏洞类型检测**: 设计多任务学习框架,同时检测多种漏洞类型。
- (2) **跨语言漏洞检测**: 在多种编程语言的代码语料上继续预训练模型,学习跨语言的通用代码表示。
- (3) **长上下文建模**: 采用稀疏注意力机制、层次化编码策略,支持更长的上下文窗口。
- (4) **数据质量提升**: 结合主动学习、弱监督学习、数据合成等技术,构建更高质量的训练数据集。
- (5) **推理加速优化**: 研究模型蒸馏、投机采样、硬件协同优化等方法,降低推理延迟。
- (6) **可解释性与可信度**: 可视化模型的注意力分布,分析漏洞检测的因果链条,为模型预测提供不确定性估计。
- (7) **集成到开发流程**: 开发与 IDE 深度集成的插件,将检测工具集成到 CI/CD 流程中。

7.5 结语

软件漏洞检测是一个具有重要理论意义和实际应用价值的研究课题。本研究围绕基于大语言模型的软件漏洞检测技术展开系统研究，提出了一系列创新方法。虽然本研究取得了一定的成果，但软件漏洞检测是一个复杂且具有挑战性的问题，仍有许多值得深入研究的方向。我们相信，随着大语言模型技术的不断进步和研究人员的持续努力，基于人工智能的漏洞检测技术将在未来发挥越来越重要的作用，为保障软件系统安全做出更大贡献。

参考文献

- [1] Cppcheck: A tool for static C/C++ code analysis[Z]. <https://cppcheck.sourceforge.io/>. 2023.
- [2] Clang Static Analyzer[Z]. <https://clang.llvm.org/docs/analyzer/checkers.html>. 2023.
- [3] Infer Static Analyzer[Z]. <https://fbinfer.com/>. 2023.
- [4] LI Z, ZOU D, XU S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.
- [5] LIN J, ZHANG Y, LI Z, et al. Cross-project transfer representation learning for software defect prediction[J]. arXiv preprint arXiv:1806.04954, 2018.
- [6] CHOI M J, JEONG S, OH H, et al. End-to-end prediction of buffer overruns from raw source code via neural memory networks[J]. arXiv preprint arXiv:1703.02458, 2017.
- [7] ZHOU Y, LIU S, SIOU J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [8] LOZHKOV A, LI R, ALLAL L B, et al. Starcoder 2 and the stack v2: The next generation[J]. arXiv preprint arXiv:2402.19173, 2024.
- [9] TONEVA M, SORDONI A, COMBES R T D, et al. An empirical study of example forgetting during deep neural network learning[J]. arXiv preprint arXiv:1812.05159, 2018.
- [10] WANG L, ZHANG X, SU H, et al. A comprehensive survey of continual learning: Theory, method and application[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024, 46(8): 5362-5383.

- [11] ALJUNDI R, LIN M, GOUJAUD B, et al. Gradient based sample selection for online continual learning[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [12] TIWARI R, KILLAMSETTY K, IYER R, et al. Gcr: Gradient coreset based replay buffer selection for continual learning[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 99-108.
- [13] JAIN R, GERVASONI N, NDHLOVU M, et al. A code centric evaluation of c/c++ vulnerability datasets for deep learning based vulnerability detection techniques[C]//Proceedings of the 16th Innovations in Software Engineering Conference. 2023: 1-10.
- [14] FAN J, LI Y, WANG S, et al. AC/C++ code vulnerability dataset with code changes and CVE summaries[C]//Proceedings of the 17th International Conference on Mining Software Repositories. 2020: 508-512.
- [15] DING Y, FU Y, IBRAHIM O, et al. Vulnerability detection with code language models: How far are we?[J]. arXiv preprint arXiv:2403.18624, 2024.
- [16] RISSE N, LIU J, BÖHME M. Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection[J]. Proceedings of the ACM on Software Engineering, 2025, 2(ISSTA): 388-410.
- [17] LI Y, YU F, WANG X. From SFT to RL: Demystifying the Post-Training Pipeline for LLM-based Vulnerability Detection[J]. arXiv preprint arXiv:2602.14012, 2026.
- [18] Claude Code Security[Z]. <https://www.anthropic.com/news/claude-code-security>. 2025.
- [19] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [20] ZHANG Y, XIE Y, LIT S, et al. Unseen horizons: Unveiling the real capability of llm code generation beyond the familiar[C]//2025 IEEE/ACM 47th International Conference on Software Engineering. 2025: 604-615.

- [21] HOULSBY N, GIURGIU A, JASTRZEBSKI S, et al. Parameter-efficient transfer learning for NLP[C]//International Conference on Machine Learning. 2019: 2790-2799.
- [22] LI X L, LIANG P. Prefix-tuning: Optimizing continuous prompts for generation [C]//Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics. 2021: 4582-4597.
- [23] HU E J, SHEN Y, WALLIS P, et al. Lora: Low-rank adaptation of large language models[J]. ICLR, 2022, 1(2): 3.
- [24] DETTMERS T, PAGNONI A, HOLTZMAN A, et al. Qlora: Efficient finetuning of quantized llms[J]. Advances in Neural Information Processing Systems, 2023, 36: 10088-10115.
- [25] KIRKPATRICK J, PASCANU R, RABINOWITZ N, et al. Overcoming catastrophic forgetting in neural networks[J]. Proceedings of the National Academy of Sciences, 2017, 114(13): 3521-3526.
- [26] CHEN H, RAZIN N, NARASIMHAN K, et al. Retaining by doing: The role of on-policy data in mitigating forgetting[J]. arXiv preprint arXiv:2510.18874, 2025.
- [27] YOON J, MADAAN D, YANG E, et al. Online coreset selection for rehearsal-based continual learning[J]. arXiv preprint arXiv:2106.01085, 2021.
- [28] LOPEZ-PAZ D, RANZATO M. Gradient episodic memory for continual learning [J]. Advances in Neural Information Processing Systems, 2017, 30.
- [29] WU T, LUO L, LI Y F, et al. Continual learning for large language models: A survey[J]. arXiv preprint arXiv:2402.01364, 2024.
- [30] WANG X, ZHANG Y, CHEN T, et al. Trace: A comprehensive benchmark for continual learning in large language models[J]. arXiv preprint arXiv:2310.06762, 2023.
- [31] SONG C, HAN X, ZENG Z, et al. Conpet: Continual parameter-efficient tuning for large language models[J]. arXiv preprint arXiv:2309.14763, 2023.

- [32] CHENG D, HUANG S, WEI F. Adapting large language models via reading comprehension[C]//The Twelfth International Conference on Learning Representations. 2023.
- [33] XUE J, CHEN X, CUI Z, et al. Learning never stops: Improving software vulnerability type identification via incremental learning[J]. Journal of Systems and Software, 2025, 230: 112544.
- [34] XIA M, MALLADI S, GURURANGAN S, et al. LESS: selecting influential data for targeted instruction tuning[C]//Proceedings of the 41st International Conference on Machine Learning. 2024: 54104-54132.
- [35] WANG J T, MITTAL P, SONG D, et al. Data Shapley in One Training Run[C]//International Conference on Learning Representations. 2025.
- [36] FERRANTE J, OTTENSTEIN K J, WARREN J D. The program dependence graph and its use in optimization[J]. ACM Transactions on Programming Languages and Systems, 1987, 9(3): 319-349.
- [37] LU J, JIANG L, LI X, et al. Towards Practical Defect-Focused Automated Code Review[C]//International Conference on Machine Learning. 2025: 40505-40536.
- [38] GUO J, WANG C, XU X, et al. RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing[C]//International Conference on Machine Learning. 2025: 21083-21100.
- [39] YU Z, ZHANG H, ZHAO Y, et al. Orcaloca: An llm agent framework for software issue localization[J]. arXiv preprint arXiv:2502.00350, 2025.
- [40] SOHRABIZADEH A, SONG J, LIU M, et al. Nemotron-cortexa: Enhancing llm agents for software engineering tasks via improved localization and solution diversity[C]//Forty-second International Conference on Machine Learning. 2025.
- [41] BINGHAM E, MANNILA H. Random projection in dimensionality reduction: applications to image and text data[C]//Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2001: 245-250.

- [42] OpenAI. GPT-4 Technical Report[J]. arXiv preprint arXiv:2303.08774, 2023.
- [43] GUO D, ZHU Q, YANG D, et al. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence[J]. arXiv preprint arXiv:2401.14196, 2024.

致 谢

时光荏苒，转眼间硕士研究生阶段的学习即将画上句号。回首在南京大学攻读学位的这段时光，心中充满感激与不舍。在此，谨向所有给予我帮助和支持的人致以最诚挚的感谢。

首先，我要衷心感谢我的导师。从论文的选题、研究方法的设计到论文的撰写与修改，导师都给予了悉心的指导。导师严谨的治学态度、渊博的专业知识和对科研工作的热情深深影响了我，使我受益终生。在导师的指导下，我不仅学会了如何做研究，更重要的是学会了如何成为一名严谨的科研工作者。

感谢实验室的各位老师和同学在科研和生活中给予我的帮助。特别是在论文研究过程中，与同学们的讨论和交流给了我很多启发，让我能够从不同角度思考问题。感谢你们营造的良好科研氛围，让我的研究生生活充实而愉快。

感谢我的家人，你们一直是最坚强的后盾。感谢父母多年来对我的养育之恩，你们的理解、支持和鼓励是我完成学业的动力源泉。没有你们的默默付出，就没有今天的我。

感谢参与本论文评审和答辩的各位专家，感谢你们在百忙之中抽出时间审阅本论文，并提出宝贵的意见和建议。

最后，感谢所有关心和支持我的人。未来的道路上，我将继续努力，不负众望。

附录 A 附录

A.1 实验数据补充

此处可放置实验中使用的详细数据表格。

A.2 源代码说明

此处可放置核心代码的说明。